

Python code from the project notebook:

```
1  # %%
2  import pandas as pd
3  import numpy as np
4  import matplotlib.pyplot as plt
5  import seaborn as sns
6  import cvxpy as cp
7  from sklearn.preprocessing import StandardScaler
8
9  # %%
10 # Font for the plots
11 plt.rcParams['font.family'] = 'Avenir'
12
13 # %% [markdown]
14 # # Data prep
15
16 # %%
17 # Load Mystery Allocation 1 (MA1) and set date as index
18 df_myst1 = pd.read_csv('Mystery Allocation 1.csv', header = None, names = ['Date', 'MA1']).set_index('Date')
19 df_myst1.index.name = None
20
21 # Load Mystery Allocation 2 (MA2) and set date as index
22 df_myst2 = pd.read_csv('Mystery Allocation 2.csv', header = None, names = ['Date', 'MA2']).set_index('Date')
23 df_myst2.index.name = None
24
25 # Join Mystery Allocations into one df on date
26 df_myst = df_myst1.join(df_myst2, how = 'inner')
27
28 # Format date index
29 df_myst.index = pd.to_datetime(df_myst.index, format="%d/%m/%Y")
30
31 df_myst.head()
32
33 # %%
34 # Load Anonymized ETFs data, drop empty 1st row and set date as index
35 df_etf = pd.read_csv('Anonymized ETFs.csv').drop(index = 0).set_index('Unnamed: 0')
36
37 # Format date index
38 df_etf.index = pd.to_datetime(df_etf.index, format="%d/%m/%Y")
39 df_etf.index.name = None
40
41 df_etf.head()
42
43 # %%
44 # Load main data, drop empty 1st row and set date as index
45 df_main = pd.read_csv('Main Asset Classes.csv', header = 4).drop(index = 0).set_index('Unnamed: 0')
46
47 # Format date index
48 df_main.index = pd.to_datetime(df_main.index, format="%d/%m/%Y")
49 df_main.index.name = None
50
51 df_main.head()
52
53 # %%
54 # Show data range across datasets
55 print('Date range for anonymised ETFs:')
56 print(df_etf.index.min())
57 print(df_etf.index.max())
58
59 print('\n')
60 print('Date range for main assets:')
61 print(df_main.index.min())
62 print(df_main.index.max())
63
64 # %%
65 # Align time indices for the same period
66 df_etf_aligned, df_main_aligned = df_etf.align(df_main, 'inner', axis=0)
67 df_myst_aligned, df_main_aligned = df_myst.align(df_main, 'inner', axis=0)
68
69 # %%
70 # Check for missing data
71 print('NaNs in ETF Data Frame:', df_etf_aligned.isna().sum().max())
72 print('NaNs in Asset Data Frame:', df_main_aligned.isna().sum().max())
73 print('NaNs in Myster Allocation:', df_myst_aligned.isna().sum().max())
74
75 # %% [markdown]
76 # # Exploratory data analysis
77
78 # %% [markdown]
79 # ### Z-Scores for main asset prices
80
81 # %%
82 scaler = StandardScaler()
83
84 # Normalise price data for comparison
85 df_zscores = pd.DataFrame(scaler.fit_transform(df_main_aligned),
86                           columns=df_main.columns,
87                           index=df_main_aligned.index)
88
89 # Define broad asset classes for comparison
```

```

90 equities = ['S&P 500', 'Nasdaq 100', 'US Small Caps', 'Euro Stoxx 50', 'UK FTSE', 'MSCI EM', 'Japan ']
91 others = list(set(df_main_aligned.columns) - set(equities))
92
93 # Plot
94 plt.figure(figsize=(12,5), dpi=200)
95 plt.plot(df_zscores[equities], color = 'blue', linewidth=0.5)
96 plt.plot(df_zscores[others], color = 'red', linewidth=0.5)
97
98 plt.title("Z-scores of Main Asset Prices", fontsize=12)
99 plt.ylabel("Z-score")
100
101 from matplotlib.lines import Line2D
102 custom_legend = [
103     Line2D([0], [0], color='blue', lw=1, label='Equities'),
104     Line2D([0], [0], color='red', lw=1, label='Other Assets')]
105
106 plt.legend(handles=custom_legend, loc='upper left', fontsize=8)
107 plt.tight_layout()
108 plt.show()
109
110 %% [markdown]
111 ### Z-scores for anonymised ETFs
112
113 %%
114 Normalise ETF prices for comparison
115 df_zscores_etf = pd.DataFrame(scaler.fit_transform(df_etf_aligned),
116                               columns=df_etf_aligned.columns,
117                               index=df_etf_aligned.index)
118
119 # Plot
120 plt.figure(figsize=(12,5), dpi=200)
121 plt.plot(df_zscores_etf, linewidth=0.5, color='black')
122 plt.title("Z-scores of Anonymised ETFs", fontsize=12)
123 plt.ylabel("Z-score")
124 plt.tight_layout()
125 plt.show()
126
127 %%
128 Identified outlier ETFs
129 outliers = ['ETF 72', 'ETF 64', 'ETF 48', 'ETF 46']
130
131 # Drop outliers in final ETF data frame
132 df_etf_aligned = df_etf.drop(columns=outliers)
133
134 # Outliers plot
135 df_etf[outliers + ['ETF 1 ']].plot() # ETF 1 for comparison
136
137 %% [markdown]
138 ### Key stats for main assets
139
140 %%
141 df_main_lreturns = np.log(df_main_aligned / df_main_aligned.shift(1)).dropna()
142 df_stats = pd.DataFrame(index=df_main_lreturns.columns)
143
144 df_stats['Mean Daily Return'] = df_main_lreturns.mean()
145 df_stats['Annualized Return'] = df_main_lreturns.mean() * 252
146 df_stats['Daily Volatility'] = df_main_lreturns.std()
147 df_stats['Annualized Volatility'] = df_main_lreturns.std() * np.sqrt(252)
148 df_stats['Sharpe Ratio'] = df_stats['Annualized Return'] / df_stats['Annualized Volatility']
149 df_stats['Max Drawdown'] = (df_main_lreturns.cumsum() - df_main_lreturns.cumsum().cummax()).min()
150 df_stats['Skewness'] = df_main_lreturns.skew()
151 df_stats['Kurtosis'] = df_main_lreturns.kurtosis()
152
153 df_stats = df_stats.round(4)
154 df_stats
155
156 %%
157 Choose key metrics to plot
158 metrics = ['Annualized Return', 'Annualized Volatility', 'Sharpe Ratio', 'Max Drawdown']
159 df_plot = df_stats[metrics]
160
161 # Setup positions for horizontal bars
162 n_assets = len(df_plot)
163 bar_height = 0.2
164 y_pos = np.arange(n_assets)
165
166 plt.figure(figsize=(11,5), dpi=200)
167
168 # Plot each metric as a horizontal bar with offset
169 for i, metric in enumerate(metrics):
170     plt.barh(y_pos + i*bar_height, df_plot[metric], height=bar_height, label=metric)
171
172 plt.yticks(y_pos + bar_height*(len(metrics)/2 - 0.5), df_plot.index)
173 plt.xlabel('Value')
174 plt.title('Asset Class Key Metrics', fontsize = 12)
175 plt.legend(fontsize=8, loc = 'upper left')
176 plt.tight_layout()
177 plt.show()
178
179 %% [markdown]
180 # Variable calculation

```

```

181
182 # %%
183 # Daily log returns
184 df_lreturns = np.log(df_etf_aligned / df_etf_aligned.shift(1)).dropna()
185 df_main_lreturns = np.log(df_main_aligned / df_main_aligned.shift(1)).dropna()
186 df_myst_lreturns = np.log(df_myst_aligned / df_myst_aligned.shift(1)).dropna()
187
188 # %%
189 # Daily cumulative log returns
190 df_lreturns_cum = df_lreturns.cumsum()
191 df_main_lreturns_cum = df_main_lreturns.cumsum()
192
193 # %%
194 # Daily mean returns
195 df_lreturns_mean = df_lreturns.mean()
196 df_main_lreturns_mean = df_main_lreturns.mean()
197
198 # %%
199 # Annualised returns
200 df_lreturns_annual = df_lreturns_mean*252
201 df_main_lreturns_annual = df_main_lreturns_mean*252
202
203 # %%
204 # Standard deviation of daily returns
205 df_vol = df_lreturns.std()
206 df_main_vol = df_main_lreturns.std()
207
208 # %%
209 # Standard deviation of annualised returns
210 df_vol_annual = df_vol * np.sqrt(252)
211 df_main_vol_annual = df_main_vol * np.sqrt(252)
212
213 # %%
214 # Sharpe ratio
215 df_sharpe_ratio = df_lreturns_annual / df_vol_annual
216 df_main_sharpe_ratio = df_main_lreturns_annual / df_main_vol_annual
217
218 # %%
219 # Sortino ratio
220
221 # Downside (negative) returns only
222 downside_returns = df_lreturns_annual[df_lreturns_annual < 0]
223
224 # Downside standard deviation
225 downside_std_annual = np.std(downside_returns) * np.sqrt(252)
226
227 # Caclulate Ratio
228 df_sortino_ratio = df_lreturns_annual / downside_std_annual
229
230 # %%
231 # Max Drawdown
232 max_drawdown = (df_lreturns_cum - df_lreturns_cum.cummax()).min().abs()
233
234 # %%
235 # Descriptive stats for key metrics
236
237 df_stats_etf = pd.DataFrame({
238     'Returns (an.)': df_lreturns_annual,
239     'Volatility (an.)': df_vol_annual,
240     'Sharpe': df_sharpe_ratio,
241     'Sortino': df_sortino_ratio,
242     'Max Drawdown': max_drawdown
243 })
244
245 df_stats_etf.describe().round(4)
246
247 # %% [markdown]
248 # # Performance & risk metrics
249
250 # %% [markdown]
251 # ### Cumulative returns (log)
252
253 # %%
254 plt.figure(figsize=(11,5), dpi=200)
255
256 plt.plot(df_lreturns_cum, color='grey', linewidth=0.8) # single color line
257 plt.plot(df_lreturns_cum.median(axis=1), color='red')
258
259 custom_legend = [Line2D([0], [0], color='red', lw=2, label='median')]
260 plt.legend(handles=custom_legend, loc='upper left', fontsize=10)
261
262 plt.title("Cumulative Log Return on ETFs", fontsize=12)
263 plt.ylabel("Log Return")
264
265 plt.grid(True, linestyle='--', alpha=0.5, axis='y')
266
267 plt.tight_layout()
268 plt.show()
269
270 # %%
271 plt.figure(figsize=(11,5), dpi=200)

```

```

272
273 sns.histplot(
274     df_lreturns_cum.iloc[-1],
275     bins=20,
276     stat='count',
277     color='blue',
278     alpha=0.4,
279     kde=True
280 )
281
282 # Calculate statistics to display
283 mean = df_lreturns_cum.iloc[-1].mean()
284 median = df_lreturns_cum.iloc[-1].median()
285 std = df_lreturns_cum.iloc[-1].std()
286
287 plt.title('Distribution of ETF Total Log Returns', )
288 plt.xlabel('Total Return')
289
290 x_min, x_max = plt.xlim()
291 plt.xticks(np.arange(np.floor(x_min*4)/4, np.ceil(x_max*4)/4 + 0.25, 0.25))
292
293 # Add vertical lines
294 plt.axvline(median, color='red', linestyle='--', linewidth=2, label='Median')
295 plt.axvline(mean, color='black', linestyle='--', linewidth=2, label='Mean')
296 plt.axvline(mean + std, color='orange', linestyle='--', linewidth=2, label='+/- 1 SD')
297 plt.axvline(mean - std, color='orange', linestyle='--', linewidth=2)
298 plt.legend(fontsize=10)
299
300
301 plt.tight_layout()
302 plt.show()
303
304 # %% [markdown]
305 # ### Risk-adjusted returns (log)
306
307 # %%
308 risk_adjusted_lreturns = (df_lreturns_cum / df_vol_annual)
309
310 plt.figure(figsize=(11,5), dpi=200)
311 plt.plot(risk_adjusted_lreturns, color='grey', linewidth=0.8)
312 plt.plot(risk_adjusted_lreturns.median(axis=1), color = 'red')
313
314 plt.title("Risk-Adjusted Cumulative Log Return", fontsize=12)
315 plt.ylabel("Risk-Adjusted Return")
316 plt.grid(True, linestyle='--', alpha=0.5, axis='y')
317
318 custom_legend = [Line2D([0], [0], color='red', lw=2, label='median')]
319 plt.legend(handles=custom_legend, loc='upper left', fontsize=10)
320
321 plt.tight_layout()
322 plt.show()
323
324 # %%
325 plt.figure(figsize=(11,5), dpi=200)
326
327 sns.histplot(
328     risk_adjusted_lreturns.iloc[-1],
329     bins=20,
330     stat='count',
331     color='blue',
332     alpha=0.4,
333     kde=True
334 )
335
336 # Calculate statistics to display
337 mean = risk_adjusted_lreturns.iloc[-1].mean()
338 median = risk_adjusted_lreturns.iloc[-1].median()
339 std = risk_adjusted_lreturns.iloc[-1].std()
340
341 plt.title('Distribution of ETF Risk-Adjusted Returns', )
342 plt.xlabel('Total Return')
343
344 x_min, x_max = plt.xlim()
345 plt.xticks(np.arange(np.floor(x_min*4)/4, np.ceil(x_max*4)/4 + 0.25, 0.25))
346
347 # Add vertical lines
348 plt.axvline(median, color='red', linestyle='--', linewidth=2, label='Median')
349 plt.axvline(mean, color='black', linestyle='--', linewidth=2, label='Mean')
350 plt.axvline(mean + std, color='orange', linestyle='--', linewidth=2, label='+/- 1 SD')
351 plt.axvline(mean - std, color='orange', linestyle='--', linewidth=2)
352 plt.legend(fontsize=10)
353
354
355 plt.tight_layout()
356 plt.show()
357
358 # %% [markdown]
359 # ### Annualised metrics
360
361 # %%
362 fig, axes = plt.subplots(1, 2, figsize=(11,5), dpi=200)

```

```

363
364 sns.histplot(
365     df_lreturns_annual,
366     bins=20,
367     stat='count',
368     color='blue',
369     alpha=0.4,
370     kde=True,
371     ax=axes[0]
372 )
373 axes[0].set_title('Annualised Log Return', fontsize = 10)
374 axes[0].set_ylabel('Count')
375
376 sns.histplot(
377     df_vol_annual,
378     bins=20,
379     stat='count',
380     color='green',
381     alpha=0.4,
382     kde=True,
383     ax=axes[1]
384 )
385 axes[1].set_title('Annualised Volatility', fontsize = 10)
386 axes[1].set_ylabel('')
387
388 plt.tight_layout()
389 plt.show()
390
391
392 # %%
393 plt.figure(figsize=(10,5),dpi=200)
394 plt.scatter(x = df_lreturns_annual, y = df_vol_annual, alpha=0.5, color='blue')
395
396 plt.xlabel('Annualised Return', fontsize = 12)
397 plt.ylabel('Annualised Volatility', fontsize = 12)
398 plt.title('Risk-Return Relationship for ETFs', fontsize = 12)
399
400 plt.tight_layout()
401 plt.show()
402
403 # %%
404 # Correlation in clusters
405
406 high_vol = df_vol_annual.loc[df_vol_annual>0.1].index
407
408 df = pd.DataFrame({
409     'vol': df_vol_annual.loc[high_vol],
410     'ret': df_lreturns_annual.loc[high_vol]
411 })
412
413 df.loc[df['ret'] < 0.17].corr()
414
415 # %%
416 fig, axes = plt.subplots(1, 2, figsize=(10,5), dpi=200)
417
418 sns.scatterplot(
419     x = df_lreturns_annual,
420     y = max_drawdown,
421     ax=axes[0],
422     c='blue',
423     alpha = 0.5
424 )
425
426 axes[0].set_ylabel('Max Drawdown')
427 axes[0].set_xlabel('Annualised Log Return')
428
429 sns.scatterplot(
430     x = df_vol_annual,
431     y = max_drawdown,
432     ax=axes[1],
433     c='blue',
434     alpha = 0.5
435 )
436
437 axes[1].set_ylabel('')
438 axes[1].set_xlabel('Annualised Volatility')
439
440 fig.suptitle('ETF Risk Reward Profile: Annualized Metrics vs Max Drawdown', fontsize=12)
441
442 plt.tight_layout()
443 plt.show()
444
445
446 # %% [markdown]
447 # ### Sharpe + Sortino
448
449 # %%
450 df_box = pd.DataFrame({
451     'Sharpe Ratio': df_sharpe_ratio,
452     'Sortino Ratio': df_sortino_ratio
453 })

```

```

454
455 # Melt the DataFrame to long format for Seaborn
456 df_melt = df_box.melt(var_name='Metric', value_name='Value')
457
458 plt.figure(figsize=(8,5), dpi=200)
459
460 sns.boxplot(x='Metric', y='Value', data=df_melt, palette='Set2')
461
462 plt.title('Distribution of ETF Performance Metrics')
463 plt.ylabel('Value')
464 plt.xlabel('')
465 plt.grid(axis='y', linestyle='--', alpha=0.7)
466 plt.tight_layout()
467 plt.show()
468
469 ##
470 fig, axes = plt.subplots(1, 2, figsize=(11,5), dpi=200)
471
472 # --- Sharpe ratio ---
473 sns.histplot(
474     df_sharpe_ratio.loc[df_sharpe_ratio.between(-2, 2)],
475     bins=20,
476     stat='count',
477     color='blue',
478     alpha=0.4,
479     kde=True,
480     ax=axes[0]
481 )
482 axes[0].set_title('Sharpe Ratio', fontsize = 10)
483 axes[0].set_ylabel('Count')
484
485 # --- Sortino ratio ---
486 sns.histplot(
487     df_sortino_ratio,
488     bins=20,
489     stat='count',
490     color='green',
491     alpha=0.4,
492     kde=True,
493     ax=axes[1]
494 )
495 axes[1].set_title('Sortino Ratio', fontsize = 10)
496 axes[1].set_ylabel('')
497
498 plt.tight_layout()
499 plt.show()
500
501 ##
502 sharpe_trimmed = df_sharpe_ratio.loc[df_sharpe_ratio.between(-2,2)]
503 sortino_trimmed = df_sortino_ratio[sharpe_trimmed.index]
504
505 plt.figure(figsize=(11,5),dpi=200)
506 plt.scatter(x = sharpe_trimmed, y = sortino_trimmed, alpha=0.5, color='blue')
507
508 min_val = min(sharpe_trimmed.min(), sortino_trimmed.min())
509 max_val = max(sharpe_trimmed.max(), sortino_trimmed.max())
510
511 # Plot 45-degree line
512 plt.plot(
513     [min_val-0.2, max_val],
514     [min_val, max_val],
515     linestyle='--',
516     color='black',
517     linewidth=1,
518     label='45 line (Sharpe = Sortino)'
519 )
520
521 custom_legend = [Line2D([0], [0], color='black', linestyle='--', lw=1, label='Sharpe = Sortino')]
522 plt.legend(handles=custom_legend, loc='upper left', fontsize=10)
523
524 plt.xlim(-0.5, 1.2)
525 plt.xlabel('Sharpe Ratio', fontsize = 10)
526 plt.ylabel('Sortino Ratio', fontsize = 10)
527 plt.title('Risk-Adjusted Performance: Sharpe vs Sortino Ratios', fontsize = 12)
528
529 plt.tight_layout()
530 plt.show()
531
532 ## [markdown]
533 ### Summary (top / bottom 5)
534
535 ##
536 df_lreturns_cum.iloc[-1].sort_values(ascending=False)
537
538 ##
539 df_vol_annual.sort_values(ascending=False)
540
541 ##
542 df_sharpe_ratio.sort_values(ascending=False)
543
544

```

```

545 # %%
546 df_sortino_ratio.sort_values(ascending=False)
547
548 # %% [markdown]
549 # # Relation analysis
550
551 # %%
552 df_combined = pd.concat([df_etf_aligned, df_main_aligned], axis=1)
553 corr_matrix = df_combined.corr().loc[df_etf_aligned.columns, df_main_aligned.columns]
554
555 # %% [markdown]
556 # ### Correlation
557
558 # %%
559 plt.figure(figsize=(20, 15))
560 sns.heatmap(corr_matrix.sort_values(by='S&P 500', ascending=False),
561             cmap="coolwarm", # red/blue diverging colors
562             center=0, # 0 = white midpoint
563             annot=False) # set to True if you want numbers
564 plt.title("Correlation Heatmap (sorted by: S&P 500)", fontsize=14)
565 plt.show()
566
567 # %% [markdown]
568 # ### Linear Regression
569
570 # %%
571 # Data Frame for both ETFs and MAs
572 df_lreturns_all = pd.concat([df_lreturns, df_myst_lreturns], axis=1)
573
574 import statsmodels.api as sm
575
576 # Empty frames to store regression parameters
577 betas = pd.DataFrame(index=df_lreturns_all.columns,
578                     columns=df_main_aligned.columns)
579
580 pvals = pd.DataFrame(index=df_lreturns_all.columns,
581                     columns=df_main_aligned.columns)
582
583 contrib = pd.DataFrame(index=df_lreturns_all.columns,
584                       columns=df_main_aligned.columns)
585
586 # Empty dictionary to store models
587 models = {}
588
589 for etf in df_lreturns_all.columns:
590     y = df_lreturns_all[etf]
591     X = sm.add_constant(df_main_aligned)
592     model = sm.OLS(y, X).fit()
593
594     # store betas, p-values (exclude constant term) and models
595     betas.loc[etf] = model.params.drop('const')
596     pvals.loc[etf] = model.pvalues.drop('const')
597     models[etf] = model
598     contrib.loc[etf] = model.params.drop('const') * df_lreturns_all[etf].sum()
599
600 # %%
601 # Filter out insignificant betas
602 significance = (pvals <= 0.05).astype(float) # 1 = significant, 0 otherwise
603 betas_signif = (betas * significance).astype(float)
604
605 # %%
606 plt.figure(figsize=(20, 15))
607 sns.heatmap(significance,
608             cmap="coolwarm", # red/blue diverging colors
609             center=0, # 0 = white midpoint
610             annot=False) # set to True if you want numbers
611 plt.title("Regression coefficients")
612 plt.show()
613
614 # %%
615 plt.figure(figsize=(20, 15))
616 sns.heatmap(betas_signif.sort_values(by=['S&P 500'], ascending=False),
617             cmap="coolwarm", # red/blue diverging colors
618             center=0, # 0 = white midpoint
619             annot=False, # set to True if you want numbers
620             vmin=-0.2,
621             vmax=0.2)
622 plt.title("Regression coefficients (significant only; trimmed colour scale; sorted by: S&P 500)", fontsize
623           =14)
624 plt.show()
625
626 # %%
627 # OLS-based contributions for a stacked barchart
628 total_returns = (contrib).sum(axis=1)
629 contrib_sorted = (contrib).loc[total_returns.sort_values(ascending=False).index]
630
631 import matplotlib.pyplot as plt
632
633 fig, ax = plt.subplots(figsize=(11,5), dpi=200)

```

```

635
636 # Stacked bar chart
637 contrib_sorted.plot(kind='bar', stacked=True, ax=ax, cmap='tab20', width=0.7)
638
639 ax.plot(
640     np.arange(len(contrib_sorted)), # z positions
641     contrib.sum(axis=1).sort_values(ascending=False), # y values
642     color='black',
643     linewidth=0.5,
644     marker='o',
645     label='Fitted Total Return'
646 )
647
648 # Labels and title
649 ax.set_ylabel("Total Log Return")
650 ax.set_title("Regression-Based Contributions of Main Assets to ETF Returns", fontsize=12)
651 plt.xticks(rotation=90, fontsize=6)
652
653 # Grid and legend
654 ax.grid(axis='y', linestyle='--', alpha=0.5)
655 ax.legend(loc = 'upper right', ncol = 2, fontsize=8)
656
657 plt.tight_layout()
658 plt.show()
659
660 ## [markdown]
661 ### PCA
662
663 ## [markdown]
664 # Principal Component Loadings
665
666 ##
667 from sklearn.decomposition import PCA
668 from sklearn.preprocessing import StandardScaler
669
670 X = df_main_lreturns.copy()
671 etf_returns = df_lreturns_all.copy()
672
673 # Standardize main asset returns
674 scaler = StandardScaler()
675 X_scaled = scaler.fit_transform(X)
676
677 # Run PCA on asset classes
678 pca = PCA()
679 factor_returns_scaled = pca.fit_transform(X_scaled)
680
681 # Create loadings DataFrame (components)
682 loadings = pd.DataFrame(
683     pca.components_,
684     columns=X.columns,
685     index=[f"PC{i+1}" for i in range(len(X.columns))]
686 )
687
688 ## [markdown]
689 # Variance explained by PCs
690
691 ##
692 explained_variance = pd.Series(pca.explained_variance_ratio_, index=loadings.index)
693
694 plt.figure(figsize=(11,5), dpi=200)
695 plt.plot(range(1, len(explained_variance)+1), explained_variance, marker='o', c='blue')
696
697 plt.xlabel("Principal Component")
698 plt.ylabel("Explained Variance Ratio")
699 plt.title("Scree Plot", fontsize = 12)
700 plt.grid(True)
701
702 plt.tight_layout()
703 plt.show()
704
705 ## [markdown]
706 # Loadings magnitude chart
707
708 ##
709 fig, ax = plt.subplots(figsize=(11,5), dpi=200)
710
711 # Plot first 5 PCs as bar chart
712 loadings.iloc[:5, :].plot(kind='bar', ax=ax, width=0.6, cmap='tab20b')
713
714 ax.set_ylabel('Loading')
715 ax.set_title('PCA Loadings: First 5 Principal Components', fontsize = 12)
716
717 plt.xticks(rotation=0)
718 ax.grid(axis='y', linestyle='--', alpha=0.5)
719
720 ax.legend(bbox_to_anchor=(1.05, 0.9), loc='upper left')
721
722 plt.tight_layout()
723 plt.show()
724
725 ## [markdown]

```



```

726 # PCA regression betas
727
728 # %%
729 etf_returns = df_lreturns_all.copy()
730
731 # Convert factor returns back to real scale
732 factor_returns_real = X.values @ pca.components_.T
733 factor_returns_real = pd.DataFrame(
734     factor_returns_real,
735     index=X.index,
736     columns=loadings.index
737 )
738
739 factor_cum = factor_returns_real.cumsum()
740
741 top_pcs = ['PC1', 'PC2', 'PC3', 'PC4', 'PC5'] # choose top 5 PCs
742
743 betas_pca = {}
744
745 for etf in etf_returns.columns:
746     y = etf_returns[etf]
747     X_f = sm.add_constant(factor_returns_real[top_pcs])
748     model = sm.OLS(y, X_f).fit()
749     betas_pca[etf] = model.params
750
751 betas_pca = pd.DataFrame(betas_pca).T
752
753 # %% [markdown]
754 # Return contributions
755
756 # %%
757 # Total cumulative factor returns
758 total_factor_return = factor_cum[top_pcs].iloc[-1] # final value per PC
759
760 # Contribution = beta * factor cumulative return
761 contrib_pca = betas_pca.drop(columns='const').multiply(total_factor_return, axis=1)
762
763 # Compute total contribution per ETF and sort
764 sorted_order = df_lreturns_cum.iloc[-1].sort_values(ascending=False).index
765 contrib_pca_sorted = contrib_pca.loc[sorted_order]
766
767 # Plot
768 fig, ax = plt.subplots(figsize=(11,5), dpi=200)
769
770 contrib_pca_sorted.plot(
771     kind='bar',
772     stacked=True,
773     ax=ax,
774     cmap='tab20',
775     width=0.8
776 )
777
778 plt.xticks(rotation=90, fontsize=6)
779
780 ax2 = ax.twinx()
781 ax2.plot(
782     np.arange(len(sorted_order)),
783     df_lreturns_cum.iloc[-1].sort_values(ascending=False).values,
784     color='black',
785     linewidth=2,
786     marker='o',
787     label='Total Return'
788 )
789
790 ax.set_title("ETF Return Attribution by PCA Factors", fontsize=12)
791 ax.set_ylabel("Contribution to Total Return")
792 ax2.set_ylabel("Total Log Return")
793
794 ax.grid(axis='y', linestyle='--', alpha=0.5)
795 ax.legend(loc='upper right', fontsize=9)
796
797
798 plt.tight_layout()
799 plt.show()
800
801 # %% [markdown]
802 # Variance contributions
803
804 # %%
805 # PC variances
806 pc_variance = factor_returns_real[top_pcs].var()
807
808 # Normalised variance contrib
809 var_contrib_raw = betas_pca[top_pcs]**2 * pc_variance
810 var_contrib_pct = var_contrib_raw.div(var_contrib_raw.sum(axis=1), axis=0)
811 sorted_order = df_lreturns_cum.iloc[-1].sort_values(ascending=False).index
812
813 # Plot
814 fig, ax = plt.subplots(figsize=(11,6), dpi=200)
815
816 var_contrib_pct.loc[sorted_order].plot(

```

```

817     kind='bar',
818     stacked=True,
819     ax=ax,
820     cmap='tab20',
821     width=0.8
822 )
823
824 ax.set_title("ETF Volatility Attribution by PCA Factors")
825 ax.set_ylabel("Contribution to Total Variance")
826 plt.xticks(rotation=90, fontsize=6)
827 ax.grid(axis='y', linestyle='--', alpha=0.5)
828 ax.legend(title='PC', bbox_to_anchor=(1.05,1), loc='upper left')
829
830 ax2 = ax.twinx()
831 ax2.plot(
832     np.arange(len(sorted_order)),
833     df_lreturns_cum.iloc[-1].sort_values(ascending=False).values,
834     color='black',
835     linewidth=2,
836     marker='o',
837     label='Total Return'
838 )
839
840 ax2.set_ylabel("Total Log Return")
841
842 plt.tight_layout()
843 plt.show()
844
845
846 # %% [markdown]
847 # Asset class identifications
848
849 # %%
850 total_returns = df_lreturns_cum.iloc[-1]
851
852 top_etfs = total_returns.nlargest(5).index
853 bot_etfs = total_returns.nsmallest(5).index
854 middle_etfs = (
855     total_returns
856     .sort_values()
857     .iloc[len(total_returns)//2 - 2 : len(total_returns)//2 + 3]
858     .index
859 )
860 mystery_etfs = ['MA1', 'MA2']
861
862 etf_groups = {
863     'Top Performing' : top_etfs,
864     'Mid Performing' : middle_etfs,
865     'Worst Performing' : bot_etfs,
866     'Mystery Allocation' : mystery_etfs
867 }
868
869 # %%
870 chosen_group = 'Worst Performing' # chose among defined above
871
872 fig, axes = plt.subplots(2, 1, figsize=(12,9), dpi=200, sharey=True)
873
874 sns.heatmap(
875     betas_pca[top_pcs].loc[etf_groups[chosen_group]],
876     center=0,
877     cmap="coolwarm",
878     annot=True,
879     fmt=".2f",
880     linewidths=0.5,
881     linecolor='white',
882     cbar_kws={'label': 'Factor Loading'},
883     ax=axes[0]
884 )
885
886 axes[0].set_title(f"Factor Loadings of {chosen_group} ETFs on Principal Components", fontsize=12)
887
888
889 sns.heatmap(betas_signif.loc[etf_groups[chosen_group]],
890             cmap="coolwarm", # red/blue diverging colors
891             center=0, # 0 = white midpoint
892             annot=True,
893             fmt=".2f",
894             linewidths=0.5,
895             linecolor='white',
896             cbar_kws={'label': 'Regression Coef.'},
897             vmin=-0.5,
898             vmax=0.5,
899             ax=axes[1])
900
901 axes[1].set_title(f"Regression Coefficients (OLS) of {chosen_group} Performing ETFs", fontsize=12)
902
903 plt.tight_layout() # Adjust layout to prevent labels from being cut off
904 plt.show()
905
906 # %% [markdown]
907 # Mystery allocations

```

```

908
909 # %% [markdown]
910 # ### Static
911
912 # %%
913 # Inputs
914 y = df_lreturns_all['MA1'].values
915 X = df_lreturns.values
916
917 n_assets = X.shape[1]
918
919 # Optimization variable
920 w = cp.Variable(n_assets)
921
922 # Objective: minimize tracking error
923 objective = cp.Minimize(cp.sum_squares(y - X @ w))
924
925 # Constraints
926 constraints = [
927     w >= 0,
928     cp.sum(w) == 1
929 ]
930
931 problem = cp.Problem(objective, constraints)
932 problem.solve()
933
934 # Results
935 weights = pd.Series(w.value, index=df_lreturns.columns)
936 weights.round(2).abs().sort_values(ascending=False).loc[weights > 0.01]
937
938 # %% [markdown]
939 # ### Dynamic
940
941 # %%
942 window = 252 # 1 year
943 assets = df_lreturns
944 etf = df_lreturns_all['MA2']
945
946 weights_rolling = []
947
948 dates = assets.index[window:]
949
950 for end in range(window, len(assets)):
951     X = assets.iloc[end-window:end].values
952     y = etf.iloc[end-window:end].values
953
954     w = cp.Variable(X.shape[1])
955
956     prob = cp.Problem(
957         cp.Minimize(cp.sum_squares(y - X @ w)),
958         [w >= 0, cp.sum(w) == 1]
959     )
960     prob.solve(solver=cp.OSQP)
961
962     weights_rolling.append(w.value)
963
964 weights_rolling = pd.DataFrame(
965     weights_rolling,
966     index=dates,
967     columns=assets.columns
968 )
969
970 # %%
971 import matplotlib.pyplot as plt
972
973 colors = plt.get_cmap('tab20c').colors # returns 20 RGB tuples
974
975 plt.figure(figsize=(11,5), dpi=200)
976
977 for i, col in enumerate(weights_rolling.columns):
978     plt.plot(weights_rolling.index, weights_rolling[col], label=col, color=colors[i % 20])
979
980 plt.title('Rolling Implied Asset Allocation For Dynamic Portfolio', fontsize=12)
981 plt.ylabel("Weight")
982 # plt.legend(loc='upper right', ncol=2, fontsize = 8)
983 plt.tight_layout()
984 plt.show()
985
986
987 # %% [markdown]
988 # Noise filtering
989
990 # %%
991 # 1. Define the approximate "flat" periods based on visual inspection
992 periods_of_interest = [
993     ('2020-03-01', '2020-10-01'), # Period 1: Greens/Brown/Lavender
994     ('2021-09-01', '2022-09-01'), # Period 2: Lavender/Green/Orange
995     ('2023-06-01', '2024-05-01') # Period 3: Violet/Blue
996 ]
997
998 # 2. Identify the Key ETFs

```

```

999 significant_etfs = set()
1000
1001 for start_date, end_date in periods_of_interest:
1002     # Slice the data for the specific period
1003     period_data = weights_rolling.loc[start_date:end_date]
1004
1005     # Calculate average allocation in this window
1006     avg_weights = period_data.mean()
1007
1008     # Select ETFs with > 5% allocation in this period
1009     leaders = avg_weights[avg_weights > 0.05].index.tolist()
1010
1011     # Add to our master list
1012     significant_etfs.update(leaders)
1013
1014     print(f"Leaders for {start_date} to {end_date}: {leaders}")
1015
1016 # 3. Filter the Main DataFrame
1017 clean_weights_rolling = weights_rolling[list(significant_etfs)]
1018
1019 # 4. Plot with DPI = 200
1020 # We create the figure and axes explicitly to control DPI
1021 fig, ax = plt.subplots(figsize=(11, 5), dpi=200)
1022
1023 clean_weights_rolling.plot(
1024     ax=ax,
1025     title="Filtered Dynamic Allocation (Dominant Regimes Only)"
1026 )
1027
1028 plt.ylabel("Weight")
1029 plt.xlabel("Date")
1030 plt.legend(loc='center right', fontsize=8)
1031 plt.tight_layout() # Adjust layout to prevent clipping of the legend
1032 plt.show()
1033
1034 # 5. Display the final list
1035 print("\nFinal Filtered ETF List:", list(significant_etfs))
1036
1037 ## [markdown]
1038 # Fitted allocation
1039
1040 ##
1041 # 1. Define the Stable Regimes
1042 # (Adjusted slightly to ensure coverage, but your dates are fine)
1043 periods_of_interest = [
1044     ('2020-03-01', '2020-10-01'),
1045     ('2021-09-01', '2022-09-01'),
1046     ('2023-06-01', '2024-05-01')
1047 ]
1048
1049 # 2. Identify the Universe of Dominant ETFs
1050 significant_etfs = set()
1051 for start, end in periods_of_interest:
1052     period_mean = weights_rolling.loc[start:end].mean()
1053     significant_etfs.update(period_mean[period_mean > 0.05].index.tolist())
1054
1055 # 3. Create the "Fitted" Series
1056 fitted_allocation = pd.DataFrame(0.0, index=weights_rolling.index, columns=list(significant_etfs))
1057
1058 print("--- FITTED ALLOCATION MODEL ---")
1059
1060 # 4. Fill with Average Weights (THE FIX)
1061 for i, (start, end) in enumerate(periods_of_interest):
1062     # Get actual data for this window
1063     period_data = weights_rolling.loc[start:end, list(significant_etfs)]
1064
1065     # Calculate the average weight
1066     avg_weights = period_data.mean()
1067     active_weights = avg_weights[avg_weights > 0.05]
1068
1069     # --- FIX START: Assign column by column to avoid broadcasting errors ---
1070     for etf, weight in active_weights.items():
1071         fitted_allocation.loc[start:end, etf] = weight
1072     # --- FIX END ---
1073
1074     print(f"\nPeriod {i+1} ({start} to {end}):")
1075     print(active_weights.round(3).to_string())
1076
1077 # 5. Fill Gaps (Optional but Recommended)
1078 # The fitted_allocation currently has 0.0s between your periods (e.g. late 2020 to late 2021).
1079 # Forward filling assumes the strategy stays static until the next regime change.
1080 fitted_allocation = fitted_allocation.replace(0.0, pd.NA).ffill().fillna(0.0)
1081
1082 # -----
1083 # CALCULATION PART
1084 # -----
1085
1086 # Align dates
1087 aligned_assets = df_lreturns.loc[fitted_allocation.index]
1088 aligned_ma2 = df_lreturns_all.loc[fitted_allocation.index, 'MA2']
1089

```

```

1090 # Calculate Reconstructed Returns
1091 # Using .sum(min_count=1) prevents 0s if data is missing, but normal sum is fine here
1092 reconstructed_returns = (fitted_allocation * aligned_assets).sum(axis=1)
1093
1094 # Calculate Cumulative Performance
1095 cum_reconstructed = reconstructed_returns.cumsum()
1096 cum_ma2 = aligned_ma2.cumsum()
1097
1098 # Plot Comparison
1099 fig, ax = plt.subplots(figsize=(11, 5), dpi=200)
1100 ax.plot(cum_ma2.index, cum_ma2, label='Original MA2 (Mystery)', color='black', linewidth=2, alpha=0.7)
1101 ax.plot(cum_reconstructed.index, cum_reconstructed, label='Reconstructed Strategy (Fitted)', color='red',
1102         linestyle='--', linewidth=1.2)
1103
1104 ax.set_title('Fitted Dynamic Allocation vs. Original MA2')
1105 ax.set_ylabel('Cumulative Log Returns')
1106 ax.legend()
1107 ax.grid(True, alpha=0.3)
1108 plt.tight_layout()
1109 plt.show()
1110
1111 # Correlation
1112 print(f"Correlation: {reconstructed_returns.corr(aligned_ma2):.4f}")
1113
1114 # %% [markdown]
1115 # Goodness-off-fit measures
1116
1117 # %%
1118 from sklearn.metrics import r2_score, mean_squared_error
1119
1120 # -----
1121 # 1. Define Metrics Function
1122 # -----
1123 def get_fit_metrics(true_returns, model_returns, model_name):
1124     # Align data to be safe
1125     common_idx = true_returns.index.intersection(model_returns.index)
1126     true_s = true_returns.loc[common_idx]
1127     model_s = model_returns.loc[common_idx]
1128
1129     # Calculate Residuals (Tracking Difference)
1130     residuals = true_s - model_s
1131
1132     # 1. Correlation
1133     corr = true_s.corr(model_s)
1134
1135     # 2. R-Squared (Goodness of Fit)
1136     r2 = r2_score(true_s, model_s)
1137
1138     # 3. RMSE (Root Mean Squared Error) - annualized (approx * sqrt(252) for scale)
1139     rmse = np.sqrt(mean_squared_error(true_s, model_s)) * np.sqrt(252)
1140
1141     # 4. Annualised Tracking Error (Std Dev of residuals)
1142     te = residuals.std() * np.sqrt(252)
1143
1144     return {
1145         "Model": model_name,
1146         "Correlation": round(corr, 4),
1147         "R-Squared": round(r2, 4),
1148         "RMSE (Ann.)": round(rmse, 4),
1149         "Tracking Error (Ann.)": round(te, 4)
1150     }
1151
1152 # -----
1153 # 2. Prepare Data
1154 # -----
1155 # A. STATIC MODEL (MA1)
1156 # Re-calculating static returns based on your 'weights' variable from earlier
1157 # (Assuming 'weights' is the Series you created in section 6.0)
1158 # If 'weights' isn't in memory, replace with your filtered top static weights
1159 static_model_returns = (df_lreturns.values @ weights.values)
1160 # Note: Ensure shapes align. If df_lreturns is (T, N) and weights is (N,), this works.
1161 # Make it a Series for easier handling
1162 static_model_series = pd.Series(static_model_returns, index=df_lreturns.index)
1163 ma1_true_series = df_lreturns_all['MA1']
1164
1165 # B. DYNAMIC MODEL (MA2) - using your new 'reconstructed_returns'
1166 # (Assuming you just ran the fixed code and have 'reconstructed_returns')
1167 dynamic_model_series = reconstructed_returns
1168 ma2_true_series = aligned_ma2
1169
1170 # -----
1171 # 3. Calculate & Display
1172 # -----
1173 metrics_list = []
1174
1175 # Calculate Static
1176 metrics_list.append(get_fit_metrics(ma1_true_series, static_model_series, "Static (MA1)"))
1177
1178 # Calculate Dynamic
1179 metrics_list.append(get_fit_metrics(ma2_true_series, dynamic_model_series, "Dynamic (MA2)"))
1180

```

```
1180 # Create DataFrame
1181 df_metrics = pd.DataFrame(metrics_list)
1182
1183 # Display
1184 print(df_metrics.to_string(index=False))
1185
1186 # Optional: Interpret the results
1187 print("\n--- Interpretation ---")
1188 print("High R    (>0.8) and Correlation (>0.9) indicate the reconstructed strategy is structurally identical.
      ")
1189 print("Low Tracking Error (<0.05) implies the 'noise' we filtered out contributed very little to risk.")
```



Aleksander Jabłoński

Student number: 12427286

Lukas Kasica

Student number: 12523206

Python for Optimisation and Finance

Project Report

Master 2 MMMEF

Paris, December 2025

TABLE OF CONTENTS

INTRODUCTION	3
1. Exploratory data analysis	3
2. Performance and risk metrics	6
2.1. Variable calculation	6
2.2. Returns	7
2.3. Risk-reward tradeoff	10
2.4. Sharpe and Sortino ratios	11
2.5. Summary classification	12
3. Relationship analysis	14
3.1. Correlation of ETFs with main asset classes	14
3.2. Linear regression based contributions.....	15
3.3. Principal Component Analysis.....	16
3.3.1. Methodology.....	16
3.3.2. Factor identification and interpretation	17
3.3.3. ETF factor exposure	19
3.4. Combined results and classification.....	20
4. Mystery allocation	22
4.1. Static portfolio.....	23
4.2. Dynamic portfolio	24
4.3. Model fit and accuracy.....	25
4.4. Implied asset allocations	26
CONCLUSION	27

INTRODUCTION

The aim of this report is to investigate a dataset of 105 anonymised Exchange-Traded Funds (ETFs) through the lens of their risk-adjusted performance, as well as to uncover their composition with respect to main asset classes, spanning broad equity indices, fixed-income instruments, and key commodities. The report highlights the relative efficiency of ETFs by considering both their return potential and risk profiles. By comparing top-performing ETFs with bottom performers, the analysis sheds light on the key characteristics that differentiate them, with particular emphasis on the role of asset class exposure, volatility, and risk management strategies in shaping ETF performance. The analysis is conducted using the Python programming language, leveraging tools from statistical inference, statistical learning, and optimisation.

The report is structured into four sections. **Section 1** presents the raw ETF and asset class data to identify the global dynamics of financial markets over the investigated period and to address potential data quality issues. It also provides a brief overview of the nature of the assets included. **Section 2** focuses on the calculation of key performance metrics, including log returns, volatility, maximum drawdown, and Sharpe and Sortino ratios. The revealed risk–reward trade-offs are discussed and ETFs are classified into best and worst performers based on these metrics. **Section 3** applies correlation, regression, and Principal Component Analysis (PCA) to explore the drivers of ETF performance by examining their relationships with the main asset classes. Finally, **Section 4** identifies two mystery portfolio allocations among the ETFs using both static and dynamic (rolling-window) quadratic optimisation. The resulting allocations are evaluated using goodness-of-fit metrics and are further analysed using the asset identification framework developed in Section 3.

1. Exploratory data analysis

The section examines key properties of the data used in the analysis and provides basic insights about the instruments included. The analysis is based on three datasets:

- ETFs - daily price data for 105 anonymised ETFs, normalised to 100 at the start;
- Main asset classes - daily price data for 14 major asset classes, including equity indices, fixed income instruments and commodities;
- Mystery allocations - daily price data for two portfolios allocated across ETFs,

The ETF dataset spans the period from 1 January 2019 to 20 May 2024, whereas the main asset dataset covers 1 January 2018 to 22 May 2024. To ensure consistency and comparability across analyses, both datasets were aligned to the common ETF date range, resulting in a unified observation window. No missing observations were identified.

Figure 1 presents the standardised price series (z-scores) of the anonymised ETFs over the sample period. The vast majority of instruments exhibit broadly similar stochastic dynamics, characterised by non-stationary price paths following shared stochastic trend. This behaviour is consistent with that of actively traded financial assets, with overall sentiment similarly influencing the performance of a wide range of assets.

Standardisation reveals the presence of four ETFs exhibiting markedly atypical behaviour. Unlike the rest of the universe, these instruments display near-linear trajectories with pronounced kinks and minimal variability over the entire five-year horizon. A closer inspection of their raw price and return series confirms that these ETFs remain almost unchanged throughout the sample period, exhibiting negligible volatility and marginal cumulative returns. Such characteristics suggest that these instruments are either illiquid, structurally constrained, or represent cash-like or synthetic products that do not reflect meaningful market exposure. As their behaviour is fundamentally inconsistent with the remainder of the ETF universe and may distort subsequent risk, correlation, and factor analyses, these four ETFs are excluded from all further work.

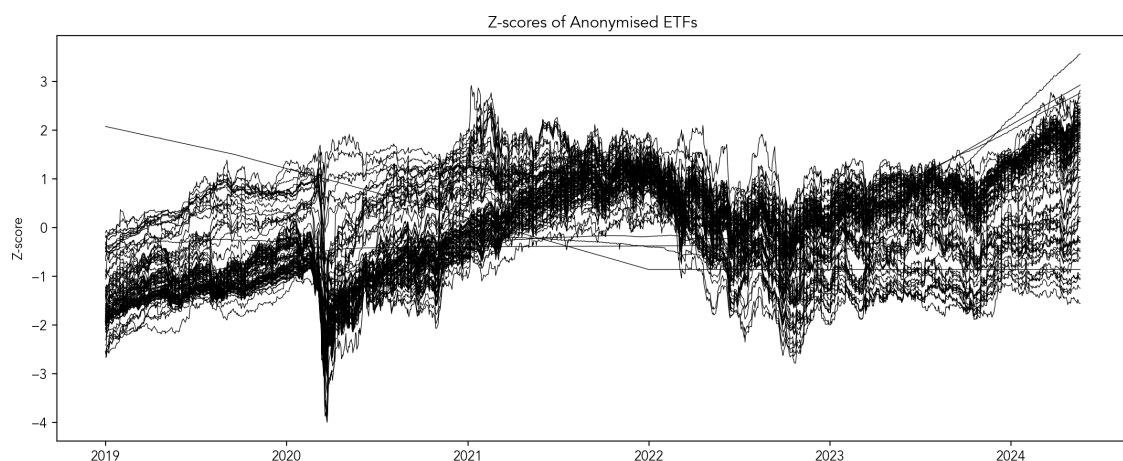


Figure 1. Z-scores for ETF daily price data.

Source: Own elaboration

Figure 2 illustrates the price dynamics of the main asset classes, while Figure 3 summarises their key risk and performance metrics. Despite differences in asset type—ranging from risk-on

equities to traditionally defensive bonds and commodities—the assets display broadly similar trend fluctuations over the sample period. The performance metrics highlight systematic differences across asset classes, with equities exhibiting both higher annualised returns and higher volatility than global bonds. Among equities, the S&P 500 and the technology-oriented Nasdaq 100 emerge as the strongest performers, while European bonds deliver the weakest returns alongside the lowest volatility. The highest risk levels are observed for US small-capitalisation equities and the Nasdaq.



Figure 2. Z-scores for daily asset prices data with key metrics.

Source: Own elaboration

2. Performance and risk metrics

2.1. Variable calculation

The analysis is conducted using daily logarithmic returns, given by:

$$r_{t,i} = \ln\left(\frac{P_{t,i}}{P_{t,i-1}}\right)$$

where $r_{t,i}$ - daily log return on i-th ETF between dates t-1 and t, computed from daily price data to ensure time-additivity and facilitate aggregation across different horizons. Log returns are particularly suitable for financial time-series analysis, as they stabilise variance and allow cumulative performance to be expressed as simple sums over time. From there we calculate ETFs' volatility by:

$$\sigma_i = \sqrt{\text{Var}(r_{t,i})}$$

where σ_i - is a standard deviation of daily log returns of i-th ETF. For annualised metrics we get:

$$r_{i,ann} = \bar{r}_i \cdot 252 \quad \sigma_{i,ann} = \sigma_i \cdot \sqrt{252}$$

where $\bar{r}_i$ is a mean daily log return of i-th ETF. We assume 252 trading days per year, in line with standard market conventions for daily financial data. Finally, we calculate two risk-reward tradeoff measures — Sharpe and Sortino ratios.

The Sharpe ratio measures the excess return per unit of total risk, capturing how efficiently an asset compensates investors for overall volatility. It treats upside and downside fluctuations symmetrically, making it most appropriate for broadly diversified portfolios. It is calculated by dividing annualised returns by annualised volatility, assuming a risk-free rate at $r_f = 0$:

$$\text{Sharpe}_i = \frac{r_{i,ann} - r_f}{\sigma_{ann}}$$

The Sortino ratio refines the Sharpe ratio by considering only downside risk (i.e. standard deviation of negative log returns: σ_i^-), penalising negative deviations from the target return while ignoring upside volatility. Like the Sharpe ratio, it is computed as the ratio of annualised returns to annualised downside deviation given zero risk-free rate, providing a more focused measure of downside-adjusted performance.

$$\text{Sortino}_i = \frac{r_{i,ann} - r_f}{\sigma_i^-}$$

2.2. Returns

Figure 3 displays cumulative sum of log returns, followed by a histogram of total performance of ETFs. Despite high correlation between ETF paths (following on average same market fluctuation like COVID-19 driven sell-off and gradual recovery).

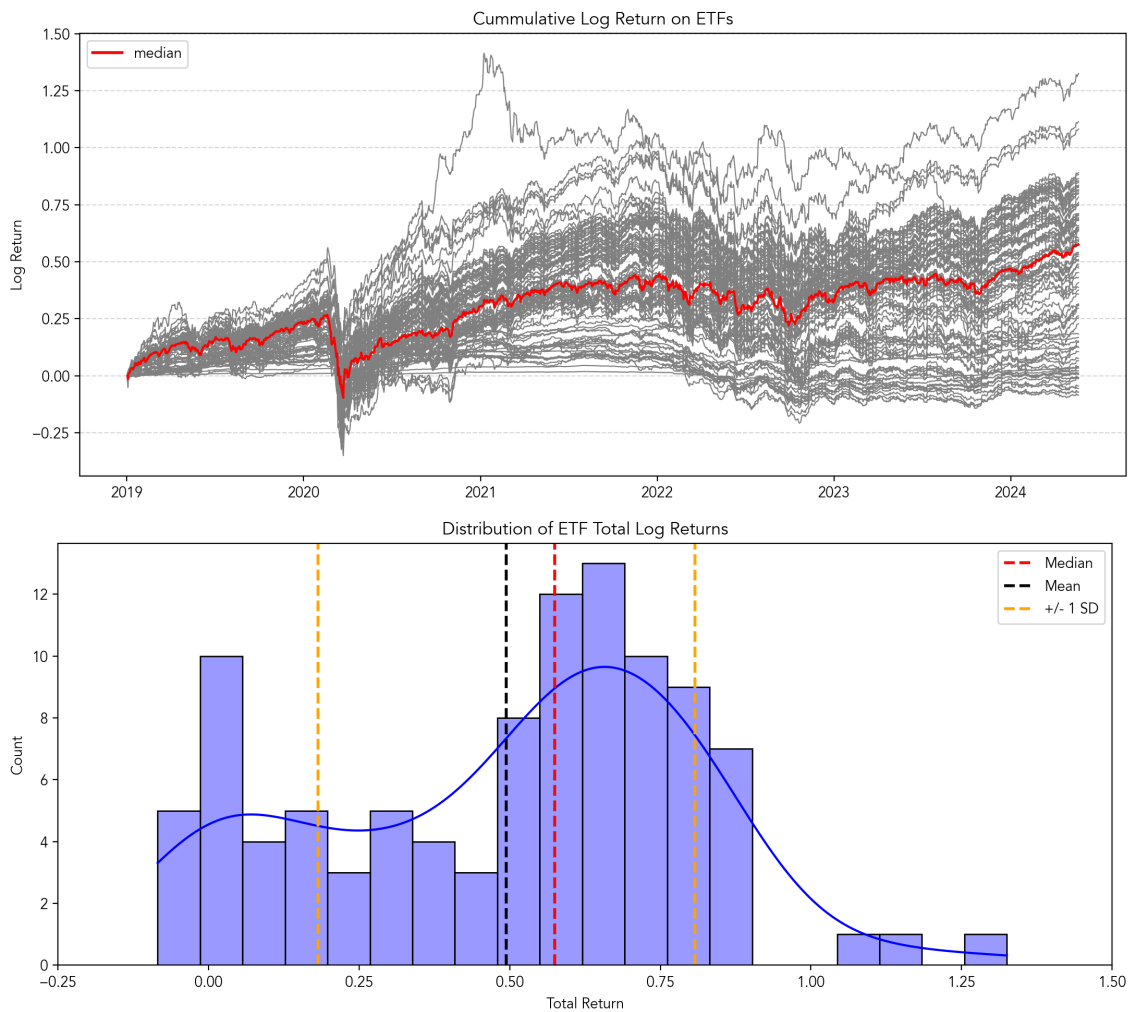


Figure 3. ETF cumulative log returns with distribution of total of returns.

Source: Own elaboration

In real terms, over the analysis period, ETF 53 was the star performer, achieving a total gain of approximately 276.3%, given by: $(e^{TotalReturn} - 1) \cdot 100\%$. ETF 12 and ETF 74 also posted stellar results, with total gains of about 204.7% and 195.0%, respectively. On the negative side, eight ETFs (indexed: 20, 39, 45, 70, 86, 90, 91, 103) posted losses over analysed period, with the worst one trading 8,13% below its initial value. The median performer achieved 77,71% return.

Table 1. Descriptive statistics for total log returns

<i>N</i>	Mean	Std	Min	25%	50%	75%	Max
101	0.494	0.313	−0.085	0.253	0.575	0.709	1.325

Source: Own elaboration

Interestingly, the histogram of total returns exhibits a bimodal pattern, with weaker-performing ETFs clustering roughly one standard deviation below the mean, while the stronger-performing assets lie slightly above the sample median. This separation suggests the presence of distinct risk/return profiles within the ETF universe, which may be driven by differences in volatility, asset class exposure, or strategy — factors that will be explored in more detail in subsequent sections.

We also examined the risk-adjusted performance of ETFs by dividing cumulative log returns by annualised volatility (Figure 4). This approach allows for a direct comparison of return efficiency, showing how much return each ETF delivers per unit of risk taken. It highlights ETFs that achieve strong performance without simply taking on the highest levels of risk. The histogram of risk-adjusted returns is much smoother and right-skewed, indicating that most ETFs deliver moderate efficiency, while a few stand out with exceptionally high risk-adjusted performance.

In this risk-adjusted framework, ETF 15 leads the ranking with a score of 5.16, outperforming even the overall top performer, ETF 53, which now ranks second with 4.93. Worth noting that ETF 15 yielded only 13.36% over the entire period. Some previously high-ranking ETFs experienced notable shifts: ETF 12, previously second, has fallen to fifth, while ETF 74, formerly third, has dropped out of the top five to eighth place. Meanwhile, new entrants have moved up in the risk-adjusted rankings: ETF 84 rises to third (from tenth) and ETF 97 climbs to fourth (from nineteenth).

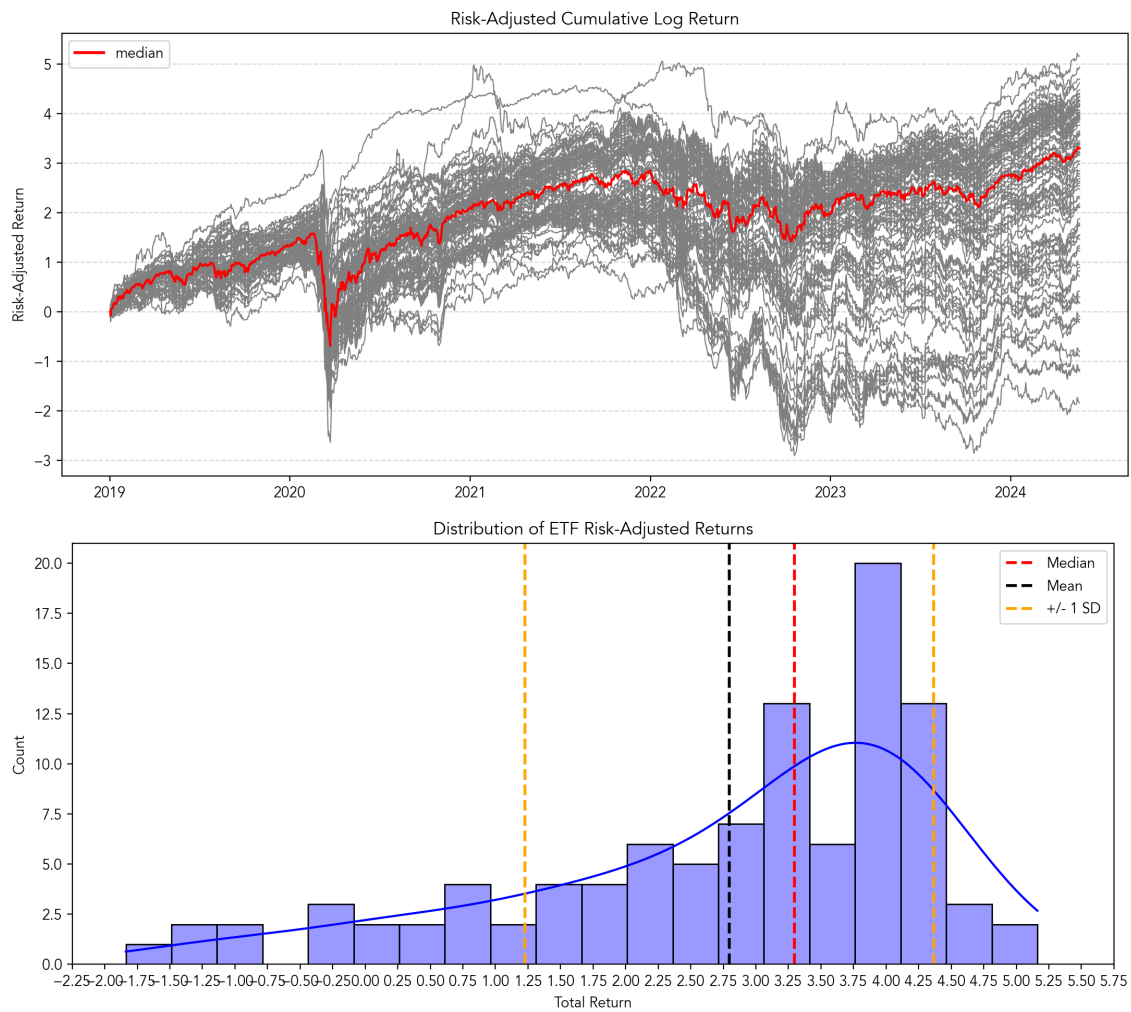


Figure 4. ETF risk-adjusted cumulative log returns with distribution of total of returns.

Source: Own elaboration

Table 2. Summary of top 5 performing ETFs

Rank	Total Return			Risk-Adjusted Return		
	ETF	Value	Total % Gain	ETF	Value	Total % Gain
1	ETF 53	1.33	276.22	ETF 15	5.16	13.36
2	ETF 12	1.11	204.65	ETF 53	4.93	276.38
3	ETF 74	1.08	195.06	ETF 84	4.70	129.68
4	ETF 23	0.89	143.51	ETF 97	4.62	113.90
5	ETF 58	0.88	141.57	ETF 12	4.48	204.72

Source: Own elaboration

2.3. Risk-reward tradeoff

The analysis of annualised metrics reveals a complex, yet intuitive relationship between risk and returns (Figure 5). The histograms show both annualised metrics exhibits a distinct bimodal distribution, even more pronounced for the volatility, with a clear cut-off between low-risk (below 0.10) and high risk clusters (between 0.15 and 0.25)

This difference in distribution is visualised in the Risk-Return scatter plot, which highlights two distinct clusters: a low-risk, low-return segment where volatility is constrained below 0.10, and a high-risk segment where volatility ranges from 0.15 to 0.25. The top 3 performing ETFs are all concentrated in the highest-volatility cluster. In this high-risk segment, but the 54% correlation between returns and volatility flags a limited ability to achieve higher returns by taking on additional volatility. If we consider only the medium cluster without top 3 ETFs, the risk-rewards correlation drops to 37%, indicating that in many case additional volatility remains unrewarded.

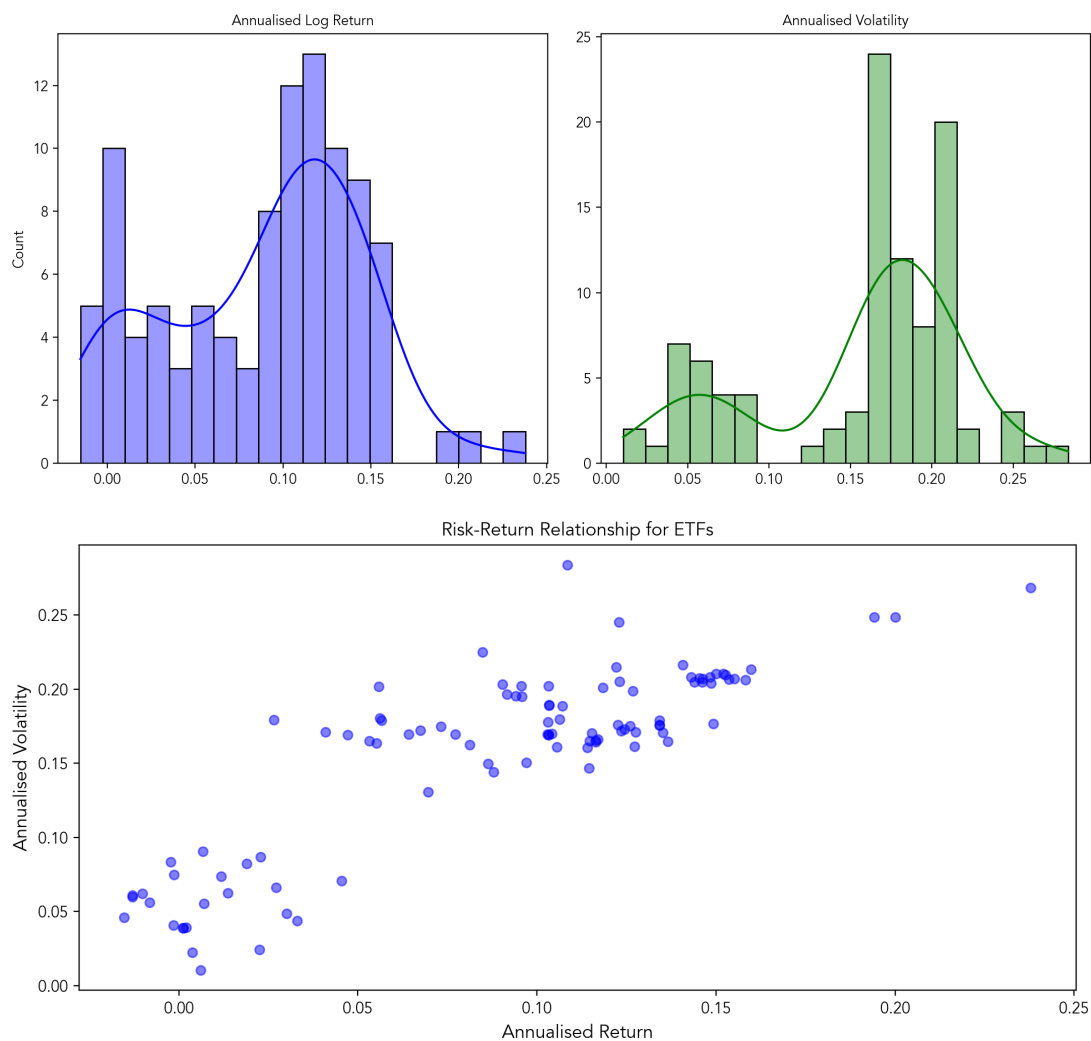


Figure 5. Annualised metrics: distribution and relationship.

Source: Own elaboration

Figure 6 relates the annualised metrics to maximum drawdown, a critical measure of downside risk. The left plot shows only a very weak positive correlation between annualised log return and max drawdown, meaning that higher average returns did not reliably equate to better downside protection. Returns largely clustered between 0.05 and 0.15 regardless of the drawdown experienced. Conversely, the right scatter plot demonstrates a strong, positive linear correlation between annualised volatility and max drawdown. ETFs with higher overall volatility almost universally experienced greater peak-to-trough losses. Volatility serves as a reliable proxy for downside risk in this ETF universe, as the safest ETFs (volatility below 0.10) maintained drawdowns below 0.20, while the most volatile funds incurred drawdowns ranging from 0.40 to over 0.90.

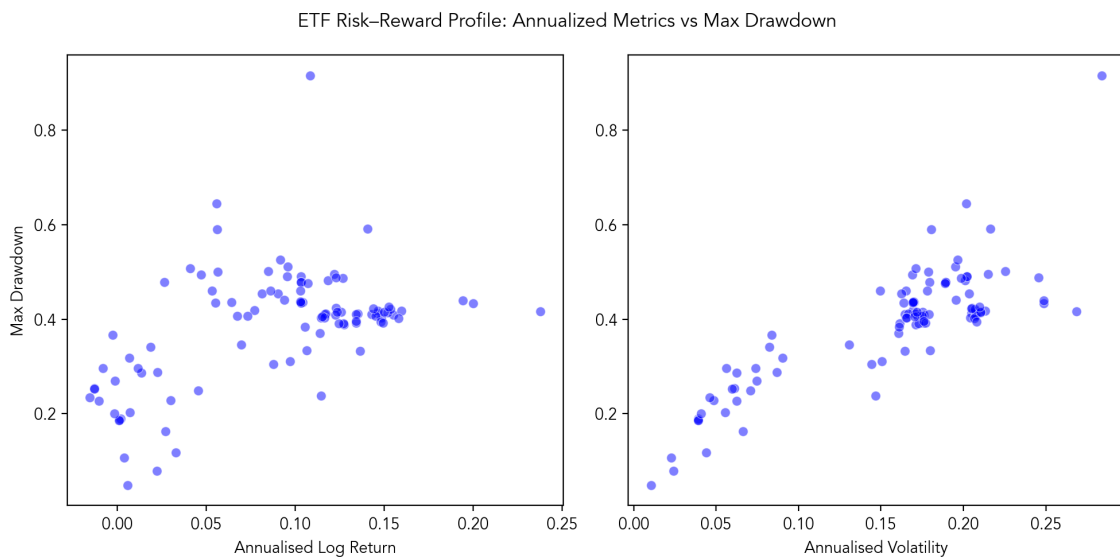


Figure 6. Max drawdown compared to annualised metrics.

Source: Own elaboration

2.4. Sharpe and Sortino ratios

The analysis of risk-adjusted returns highlights a critical divergence between total risk and downside risk (Figure 7). The histograms show that while the Sharpe Ratio is slightly skewed towards higher values (median around 0.6), the Sortino Ratio exhibits a distinct bimodal distribution with a higher central tendency (median around 1.4), distinguishing clearly between lower-efficiency funds and a larger cluster of high-efficiency performers.

The scatter plot further clarifies this relationship by comparing each ETF against the 45° diagonal. Methodologically, ETFs lying close to this line would exhibit similar Sharpe and Sortino ratios, indicating downside risk comparable to total volatility. However, the vast majority of ETFs

in this universe are located significantly above the diagonal, where the Sortino Ratio exceeds the Sharpe Ratio. This suggests that for these funds, a large portion of their total volatility stems from positive upside variance rather than downside risk. Conversely, only a few ETFs fall below the diagonal—a zone that would indicate pronounced negative return episodes—confirming that the observed volatility was largely "good" volatility associated with upward price movements.

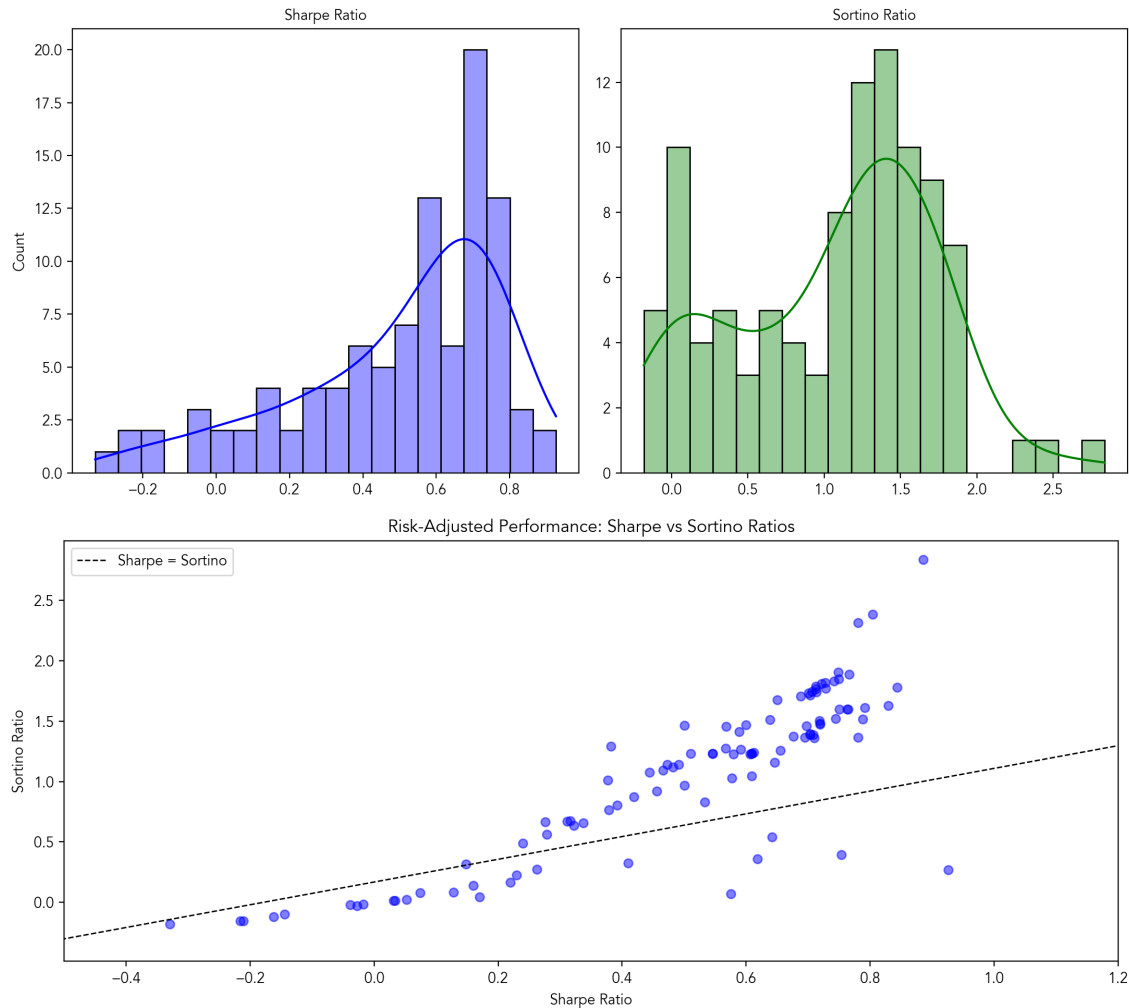


Figure 7. Sharpe vs Sortino: distribution and relationship.

Source: Own elaboration

2.5. Summary classification

Table 3 contains the top and bottom 5 ETFs for selected risk and performance metrics. The ranking highlights a clear distinction between aggressive growth drivers and highly efficient, low-volatility assets. ETF 53 and ETF 12 dominate the growth category, ranking as the top two funds for Cumulative Log Return and the Sortino Ratio. However, they also appear in the top five for Annualised Volatility, indicating their superior performance required accepting significant total risk.

In contrast, ETF 15 achieves the highest Sharpe Ratio (0.93) without chasing high raw returns, but rather by maintaining one of the lowest annualised volatilities (0.02) in the entire universe, effectively identifying it as the most efficient "safe haven" asset. Conversely, the bottom of the table is remarkably consistent: ETF 91, ETF 70, and ETF 39 rank as the worst performers across nearly every metric, offering negative nominal returns alongside the lowest risk-adjusted efficiency.

Table 3. Summary classification of ETFs by chosen metrics

Metric	Top 5	Bottom 5
Cumulative Log Return <i>(Total Growth)</i>	ETF 53 (1.33) ETF 12 (1.11) ETF 74 (1.08) ETF 23 (0.89) ETF 58 (0.88)	ETF 91 (-0.08) ETF 70 (-0.07) ETF 39 (-0.07) ETF 90 (-0.06) ETF 20 (-0.05)
Annualized Volatility <i>(Highest vs. Lowest Risk)</i>	ETF 13 (0.28) ETF 53 (0.27) ETF 74 (0.25) ETF 12 (0.25) ETF 75 (0.25)	ETF 65 (0.01) ETF 43 (0.02) ETF 15 (0.02) ETF 78 (0.04) ETF 57 (0.04)
Sharpe Ratio <i>(Risk-Adjusted Return)</i>	ETF 15 (0.93) ETF 53 (0.89) ETF 84 (0.84) ETF 97 (0.83) ETF 12 (0.80)	ETF 91 (-0.33) ETF 70 (-0.22) ETF 39 (-0.21) ETF 90 (-0.16) ETF 20 (-0.14)
Sortino Ratio <i>(Downside Efficiency)</i>	ETF 53 (2.84) ETF 12 (2.38) ETF 74 (2.31) ETF 23 (1.90) ETF 58 (1.89)	ETF 91 (-0.18) ETF 70 (-0.15) ETF 39 (-0.15) ETF 90 (-0.12) ETF 20 (-0.10)

Source: Own elaboration

3. Relationship analysis

3.1. Correlation of ETFs with main asset classes

The correlation heatmap between the 100 ETFs and major global asset classes reveals a highly polarised investment universe. The upper section of the map is characterised by Equity Dominance, where a large cluster of "Pure Equity" funds exhibits strong positive correlations (dark red) with Developed Market indices (S&P 500, Nasdaq 100, Euro Stoxx 50) and High Yield credit, confirming that a significant portion of the sample is driven by aggressive market beta. Conversely, the bottom section defines a clear Fixed Income / Low-Risk path, where ETFs show high correlations with Investment Grade bonds (US IG, EU IG) and sovereign debt, while often displaying negative or low correlations with equities. Sandwiched between these two distinct blocks is vague middle strip, representing broadly diversified or multi-asset portfolios. In this transitional zone, correlations are moderate (lighter orange/red) across both asset types, indicating that true "all-weather" or balanced strategies are less prevalent in this dataset than pure directional bets.

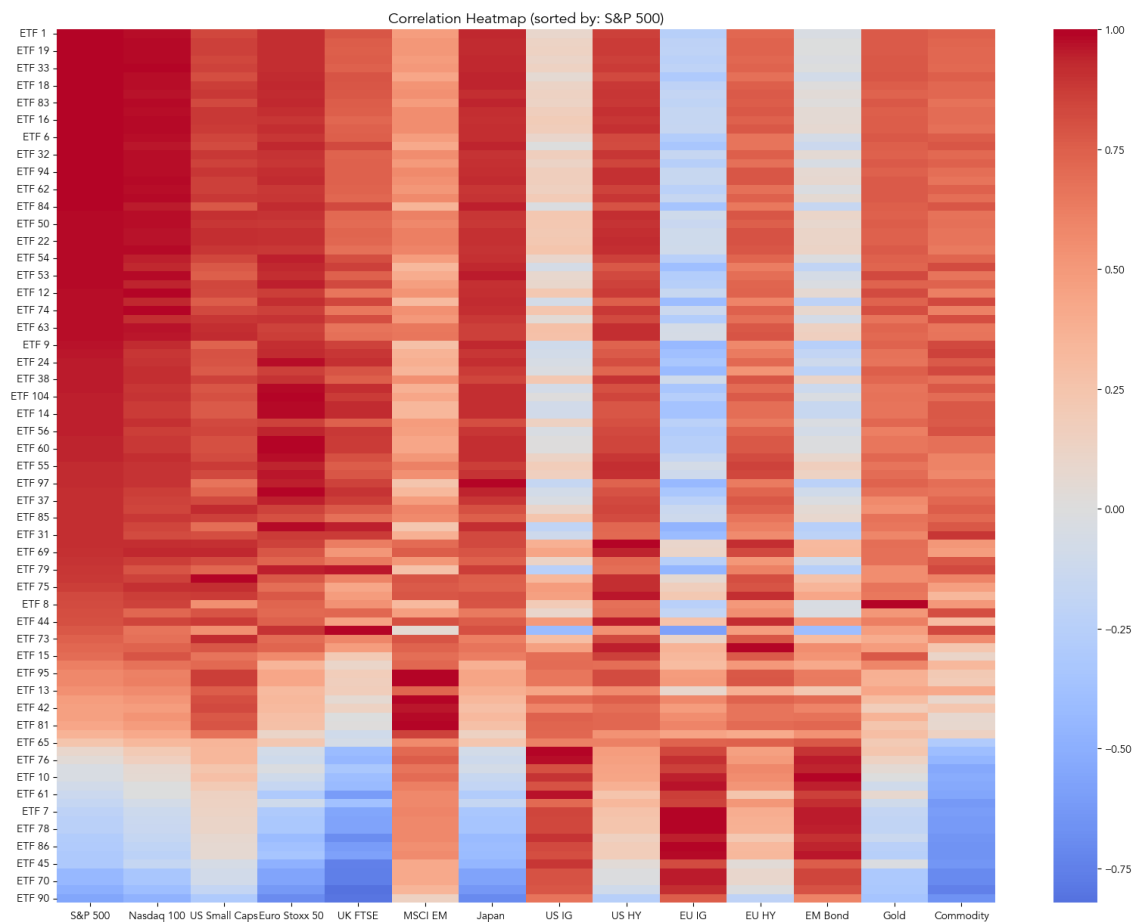


Figure 8. Heatmap with correlations between ETFs and main asset classes.

Source: Own elaboration

3.2. Linear regression based contributions

To extend the analysis beyond simple point-wise correlations, we estimated a Multiple Linear Regression (MLR) model for each ETF to isolate the marginal contribution of each asset class to performance. The model equation is as follows:

$$r_{t,i} = \alpha + \Gamma \mathbf{X}_t + \epsilon_t$$

where daily log return of the i -th ETF at time t is regressed on the matrix $\mathbf{X}$, comprising the daily log returns of the main asset classes. The use of log returns ensures stationarity and constrained variance, thereby satisfying the core stability assumptions required by the Gauss-Markov theorem for the Ordinary Least Squares estimator.

The resulting matrix of statistically significant regression coefficients confirms and sharpens the polarised market structure identified previously. The upper section of the universe exhibits a distinct Equity Beta profile, characterised by large, positive, and statistically significant coefficients for the S&P 500, Nasdaq 100, and Euro Stoxx 50, indicating that these funds are essentially leveraged plays on developed equity markets. Conversely, the lower section reveals a clear Credit/Duration cluster, where equity betas vanish and are replaced by significant positive loadings on US and EU Investment Grade and High Yield bond, as well as more geographically diversified equities. This clear separation in significant coefficients validates that the majority of these ETFs are driven by distinct, non-overlapping primary risk factors—either global equity risk or credit spread/interest rate risk—with little evidence of complex, multi-asset alpha generation in the middle of the distribution.

The decomposition of returns via OLS contributions (Figure 9) provides a critical link to our earlier risk-reward analysis, revealing clear segmentation in performance drivers. Top-tier performance was predominantly fueled by US equities, with specific tilts toward high-beta growth and small-cap exposures, while the middle tier relied on diversified global value stocks and the lower tail was dominated by bond-heavy portfolios. However, these results must be interpreted with caution due to the significant multicollinearity among asset classes (e.g., S&P 500 and Nasdaq 100), which can distort the signs of individual coefficients. Despite this statistical limitation, the model robustly confirms the fundamental trade-off: superior nominal returns in this universe were

inextricably linked to accepting the higher volatility inherent to equity markets compared to fixed income.

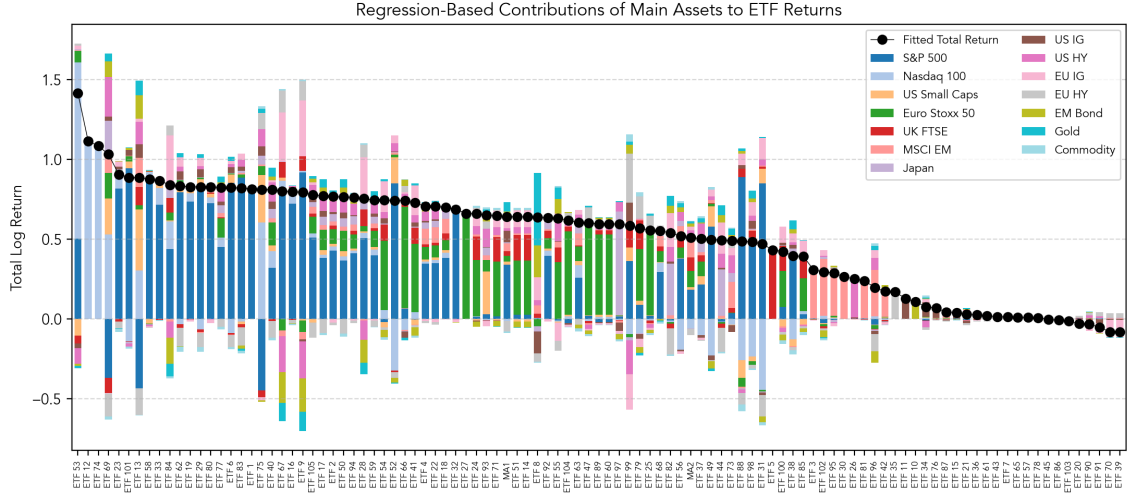


Figure 9. OLS-based contributions to ETF performance.

Source: Own elaboration

3.3. Principal Component Analysis

3.3.1. Methodology

To resolve the multicollinearity issue inherent in the OLS framework and identify orthogonal drivers of risk and return, we employ Principal Component Analysis (PCA). PCA is a dimensionality reduction technique that transforms the original set of correlated variables (asset class returns) into a new set of uncorrelated variables called principal components.

Mathematically, let X be the $T \times N$ matrix of standardised daily log returns for the $N = 14$ asset classes over T time periods. The first step involves computing the covariance matrix (or correlation matrix, given standardisation):

$$\Sigma = \frac{1}{T-1} X^T X$$

We then perform an decomposition of the covariance matrix to solve for the eigenvalues λ and eigenvectors v : $\Sigma v = \lambda v$.

The eigenvectors, sorted by the magnitude of their corresponding eigenvalues, represent the Principal Components or "loadings." These vectors define the directions of maximum variance in the data. The eigenvalues represent the amount of variance explained by each corresponding component. The first principal component (PC1) accounts for the largest possible variance, the second (PC2) accounts for the largest remaining variance while being orthogonal to the first, and so on.

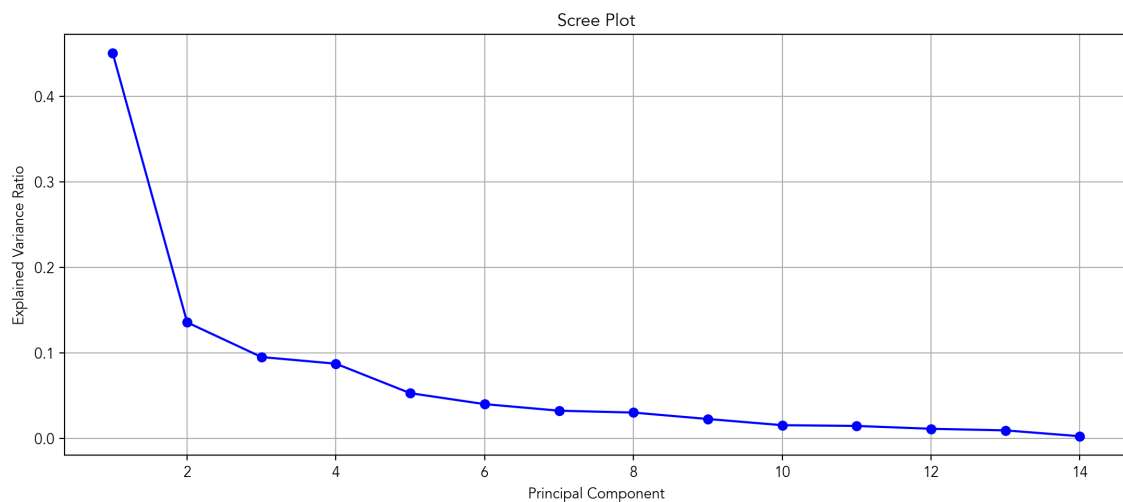


Figure 10. Scree plot for Principal Components.

Source: Own elaboration

3.3.2. Factor identification and interpretation

The selection of the optimal number of factors is guided by the Scree Plot (Figure 10), which displays the proportion of total variance explained by each successive principal component. The plot reveals a dominant first component (PC1) that alone accounts for over 45% of the total variance, reflecting the high degree of correlation across global financial markets. The "elbow" of the curve—the point where the marginal gain in explained variance diminishes significantly—appears after the fifth component. Consequently, we retain the first five Principal Components (PC1–PC5) for analysis, as they collectively capture the vast majority of the systematic risk structure while filtering out idiosyncratic noise.

To interpret the financial meaning of these latent factors, we examine the Principal Component Loadings. This chart illustrates the correlation between each original asset class and the new orthogonal components. By analysing the magnitude and sign of the loadings (the height of the bars), we assign the following distinct economic profiles to the five retained components:

- **PC1 (Global Market Beta):** Characterised by uniformly positive loadings across almost all risky assets—particularly equities (S&P 500, Euro Stoxx 50) and High Yield credit—this factor represents the broad "risk-on" market sentiment. It drives the synchronous movement of global capital markets.
- **PC2 (Credit/Duration):** This factor captures the classic "flight to safety" dynamic. It exhibits strong positive loadings on defensive assets like Investment Grade bonds (US IG, EU IG) and Gold, contrasted against negative loadings on major equity indices, representing the trade-off between safety/duration and equity risk.
- **PC3 (International vs US):** A geographic differentiator that separates the performance of US indices (negative loadings on Nasdaq 100/S&P 500) from International and Emerging Markets (positive loadings on Japan, MSCI EM).
- **PC4 (Real Assets/Inflation):** This factor is distinct and specialised, driven almost exclusively by very high positive loadings on Commodities and Gold, serving as a proxy for inflation hedging or raw material cycles.
- **PC5 (Growth vs. Value):** A style-based factor that contrasts high-growth, tech-heavy markets (positive loadings on Nasdaq 100 and Japan) against traditional, value-heavy European indices (negative loadings on UK FTSE, Euro Stoxx 50).

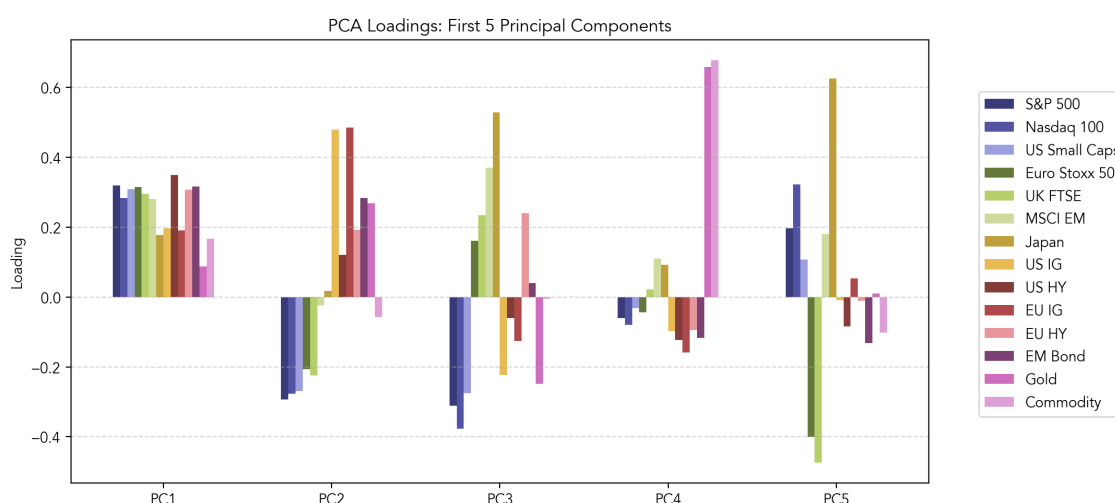


Figure 11. PC loadings per asset class.

Source: Own elaboration

3.3.3. ETF factor exposure

With the economic identity of the five Principal Components established, the final analytical step is to determine the specific sensitivity of each ETF to these orthogonal risk drivers. Unlike the initial asset-class regression, which suffered from multicollinearity, projecting ETF returns onto the uncorrelated Principal Components allows for a robust and stable attribution of risk. We achieve this by treating the derived Principal Components as independent risk factors in a multi-factor model. Formally, for each ETF i , we estimate the following time-series regression equation:

$$R_{i,t} = \alpha_i + \beta_{i,1}PC_{1,t} + \beta_{i,2}PC_{2,t} + \beta_{i,3}PC_{3,t} + \beta_{i,4}PC_{4,t} + \beta_{i,5}PC_{5,t} + \epsilon_{i,t}$$

where:

- $R_{i,t}$ is the standardised daily log return of i -th ETF at time t ;
- $PC_{k,t}$ is the return of the k -th Principal Component (factor) at time t ;
- α_i represents the unexplained intercept (or idiosyncratic drift);
- $\beta_{i,k}$ is the PCA Beta or factor loading, measuring the sensitivity of the ETF to the k -th factor.

The resulting PCA Betas provide a precise "genetic code" for each fund. A high positive $\beta_{i,1}$ indicates a fund that rallies strongly with global risk sentiment, while a significant negative $\beta_{i,5}$ would identify a fund inversely correlated with the Growth/Tech factor (i.e., a value-focused fund).

Figure 13 displays the PCA-based attribution of total ETF returns, using factor betas to calculate cumulative contributions. It is evident that the top-performing ETFs benefited from a potent combination of broad market exposure (PC1) fused with a US-bias (PC3) and a strong Growth/Tech focus (positive PC5), while maintaining minimal hedging via bond exposure. The middle segment of the distribution, comprising average-performing and generally lower-risk funds, is characterised by a pronounced negative PC5 contribution, signalling a distinct Value focus. Finally, the weakest ETFs underperformed mainly due to high fixed income exposure, indicated by large negative contributions from the Credit/Duration factor (PC2).

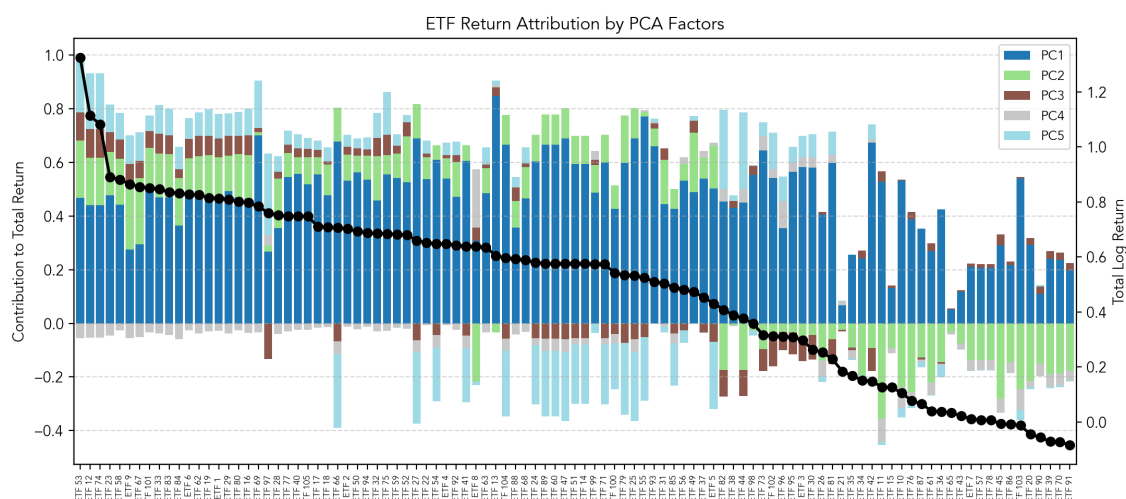


Figure 12. PCA-based contributions to the total return.

Source: Own elaboration

3.4. Combined results and classification

The combined analysis using both OLS regression and PCA factor attribution unequivocally identifies structural asset allocation. To pinpoint the structural drivers of performance, we synthesise findings from both OLS regression and PCA factor attribution into a unified forensic analysis. Figure 14 visualises this comparison for the top, median, and bottom five ETFs, revealing the distinct asset allocation profiles that defined each performance tier.

The Top Performing ETFs (e.g., ETF 53, ETF 12) were driven by a concentrated "pure play" exposure to aggressive US equity risk. OLS results show near-unitary sensitivity to the Nasdaq 100 and S&P 500, with absolutely no defensive hedging. This aggressive positioning is confirmed by PCA, where positive loadings on PC1 (Global Beta) and negative loadings on PC2 (Credit) and PC3 (International) reveal a strategy that systematically shorted safety and diversification. Crucially, their returns were amplified by a high positive loading on PC5, confirming that their outperformance was fueled specifically by the Growth/Tech factor rather than generic market beta.

The Median Performing ETFs (e.g., ETF 99, ETF 14) reveal a structural bias distinct from both the leaders and laggards. Their OLS coefficients shift away from the US, dominated instead by Euro Stoxx 50 and UK FTSE, identifying them as primarily European equity portfolios. PCA mathematically isolates this geographic displacement: these funds combine positive loadings on PC3 (International) with a distinct negative sensitivity to PC5 (Value). Consequently, their "average" results reflect the opportunity cost of holding equities that missed the US tech premium, placing them on the wrong side of the decade's primary AI-trade.

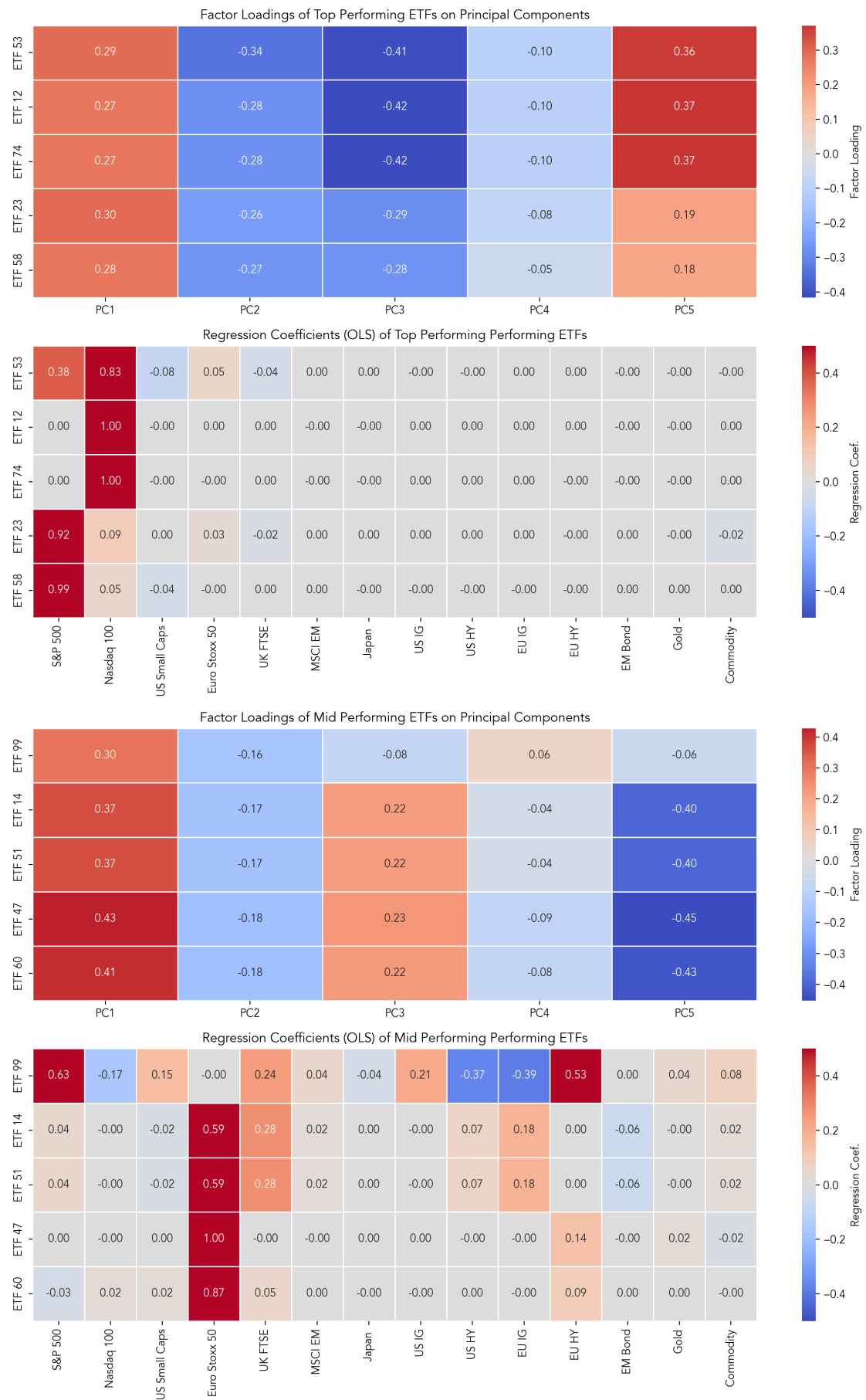


Figure 13. Hybrid identification of 5 Top and Mid performing ETFs.

Source: Own elaboration

The Worst Performing ETFs (e.g., ETF 91, ETF 70) underperformed due to structural asset allocation rather than selection error. OLS analysis exposes a stark "bond-heavy" DNA, with statistically significant loadings on Investment Grade bonds and virtually zero equity exposure. This is robustly corroborated by PCA, where these funds exhibit their highest sensitivity to PC2 (Credit/Duration)—the defensive "flight-to-safety" factor—while displaying negligible exposure to the high-growth PC5 factor. Ultimately, these portfolios paid the systemic price of being positioned as "safe havens" in a market that overwhelmingly rewarded risk.

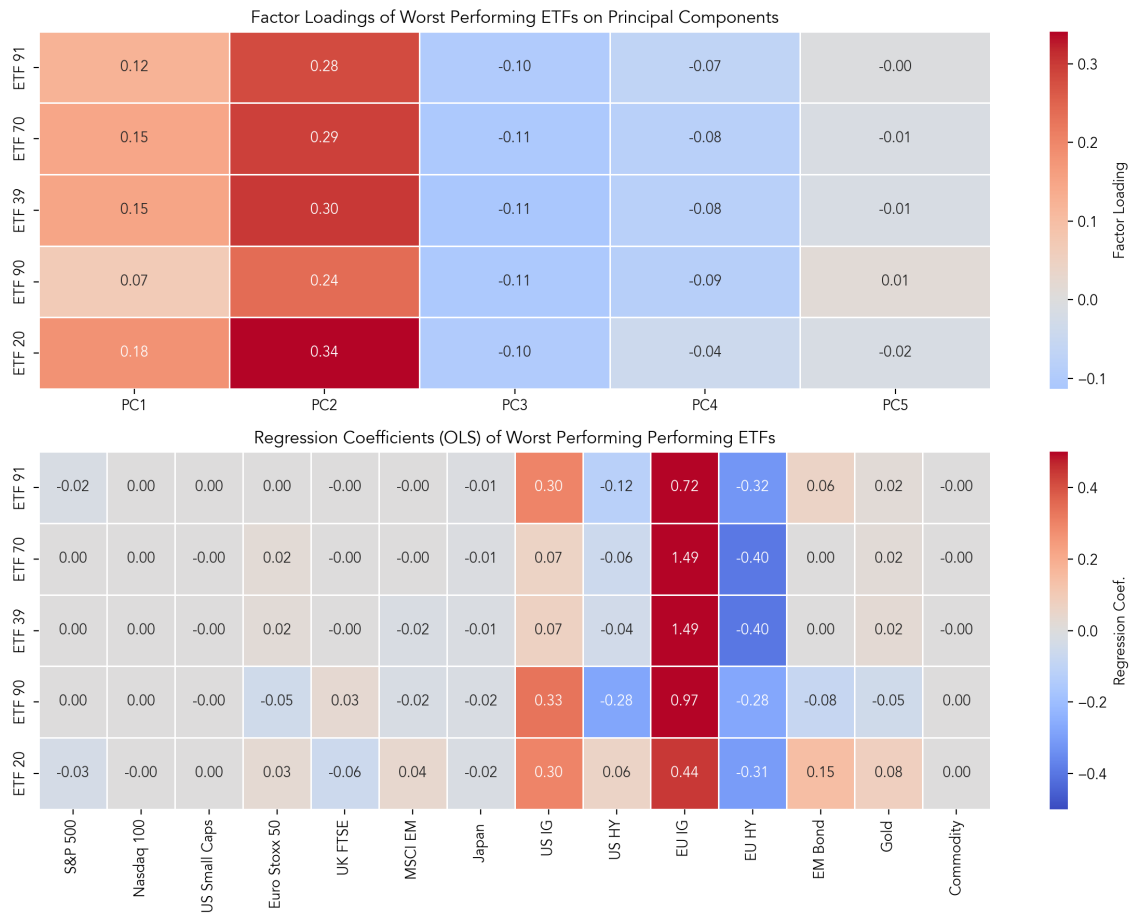


Figure 14. Hybrid identification of 5 Worst performing ETFs.

Source: Own elaboration

4. Mystery allocation

To reverse-engineer the composition of the two unidentified portfolios (MA1 and MA2), we employ Constrained Least Squares Optimisation. This technique solves for the optimal vector of weights that minimises the tracking error (sum of squared return deviations) between the mystery portfolio and our available investment universe, subject to "long-only" and "fully invested" constraints.

Formally, we define the mystery portfolio's return at time t as $R_{M,t}$ and the universe of $N=101$ available ETFs ($R_{j,t}$). The objective is to find the vector of weights $\omega = [\omega_1, \dots, \omega_N]^\top$ that minimises the sum of squared residuals over the observation period T , subject to long-only and fully-invested constraints:

$$\begin{aligned} &\underset{\omega}{\text{minimize}} && \sum_{t=1}^T \left(R_{M,t} - \sum_{j=1}^N \omega_j R_{j,t} \right)^2 \\ &\text{subject to} && \sum_{j=1}^N \omega_j = 1, \\ &&& \omega_j \geq 0, \quad \forall j = 1, \dots, N. \end{aligned}$$

4.1. Static portfolio

For the static portfolio, we solve for a single global optimisation problem, with constant set of weights over the entire five-year period. The constrained optimisation for MA1 reveals a highly concentrated portfolio structure, heavily reliant on a few key drivers. The top three positions—ETF 31 (20%), ETF 40 (18%), and ETF 53 (15%)—collectively command over 53% of the total capital. The remaining allocation is distributed across a "long tail" of smaller positions ranging from 10% to 2% (e.g., ETF 8, ETF 11). This distribution suggests a "Core-Satellite" strategy, where the portfolio performance is primarily dictated by three high-conviction bets, supplemented by smaller, likely diversifying or tactical exposures.

Table 4. Optimised weights for MA1 composition (filtered by: 1% <).

	weight (rounded)
ETF 31	0.20
ETF 40	0.18
ETF 53	0.15
ETF 8	0.10
ETF 11	0.10
ETF 10	0.05
ETF 102	0.05
ETF 5	0.05
ETF 34	0.05
ETF 1	0.04
ETF 47	0.02

Source: Own elaboration

4.2. Dynamic portfolio

For the time-varying portfolio, we utilise a Rolling Constrained Optimisation with a 252-day look-back window. This iteratively solves for the optimal weights at each time step based on the most recent year of data. The initial rolling optimisation for the dynamic portfolio (MA2) produced a volatile output, characterised by frequent shifts and significant noise allocations (Figure 15). To isolate the true tactical intent of the MA2, we applied a Regime-Based Filtering Rule. By visually inspecting the allocation timeline, we identified three distinct periods of stability: "Period 1" (Feb 2020 – Oct 2020), "Period 2" (Aug 2021 – Oct 2022), and "Period 3" (Nov 2023 – May 2024). We then filtered the dataset to retain only those ETFs that maintained a dominant average weight (greater than 5%) within at least one of these stable windows. This process successfully revealed a sequence of higher-conviction bets, allowing for a precise reconstruction of the strategy. Figure 16 displays the cumulative performance of the fitted dynamic allocation against the MA2.

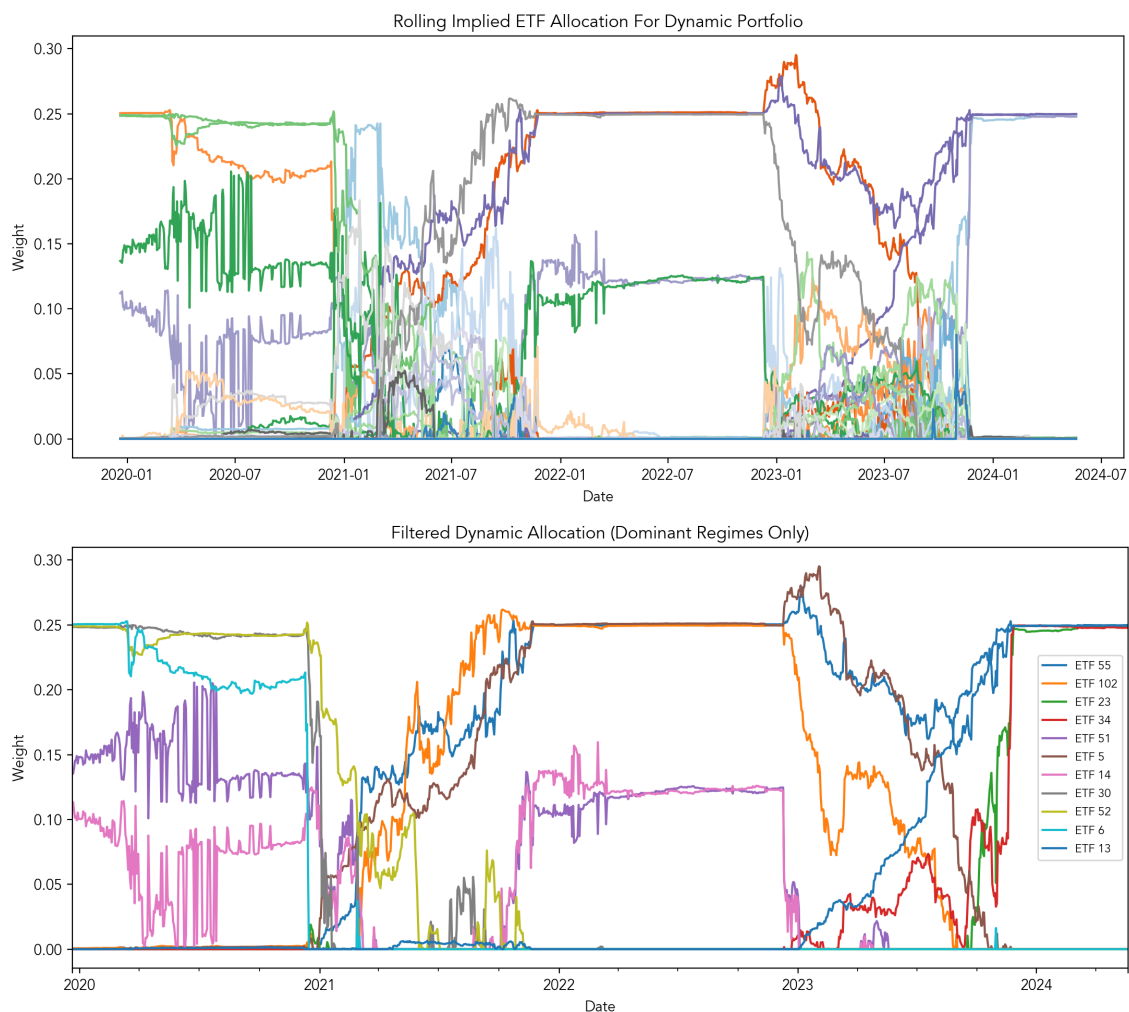


Figure 15. Dynamic weights for MA2: all vs filtered.

Source: Own elaboration

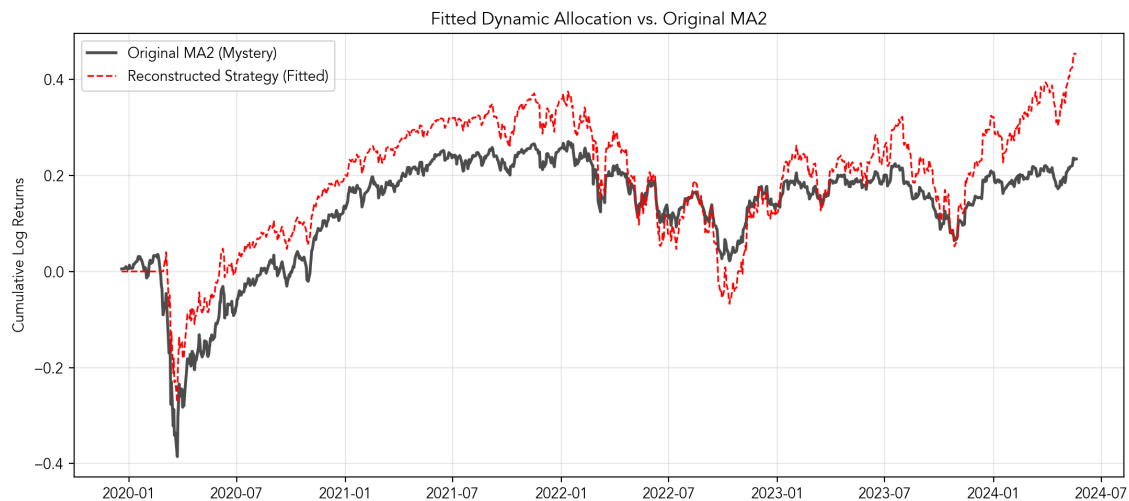


Figure 16. Fitted MA2.

Source: Own elaboration

4.3. Model fit and accuracy

To validate our reconstruction, we calculated key error metrics comparing our replicated portfolios against the original mystery series. As shown in the table below, the Static (MA1) model achieves a near-perfect fit (R^2 virtually at 1), confirming that the mystery portfolio is indeed a constant-mix strategy comprised of the identified ETFs.

The Dynamic (MA2) reconstruction, while less precise due to the regime-based simplification, still demonstrates strong explanatory power. A correlation of 0.90 confirms that our "Regime-Switching" model successfully captures the strategy's primary tactical moves, even after filtering out the optimiser's daily noise.

Table 5. Goodness-of-fit metrics for identified allocations.

Model	Correlation	R^2	RMSE (Ann.)	Tracking Error (Ann.)
Static (MA1)	1.000	0.9999	0.0011	0.0010
Dynamic (MA2)	0.896	0.7115	0.0941	0.0941

Source: Own elaboration

4.4. Implied asset allocations

Additionally, we undergo the Mystery Allocations through our hybrid OLS-PCA treatment to identify the asset classes underlying the composition of portfolios (Figure 17). The analysis exposes MA1 as a diversified, constant-mix strategy with stable loadings on both broad equity (S&P 500) and fixed income factors (US and EU IG, Gold), consistent with a balanced "Core" profile. Conversely, MA2 reveals a heavier equity focused, diversified between US and global markets, while avoiding overly growth-oriented stocks. This is further underlined once we run the dynamic optimisation on assets classes instead of ETFs (Figure 18). First period is dominated by S&P 500, with MSCI EM and Euro Stoxx 50 coming right after, second gives priority to Europe and EM equities with no US exposure and third becomes the most diversified, with Japan and Nasdaq taking the lead.

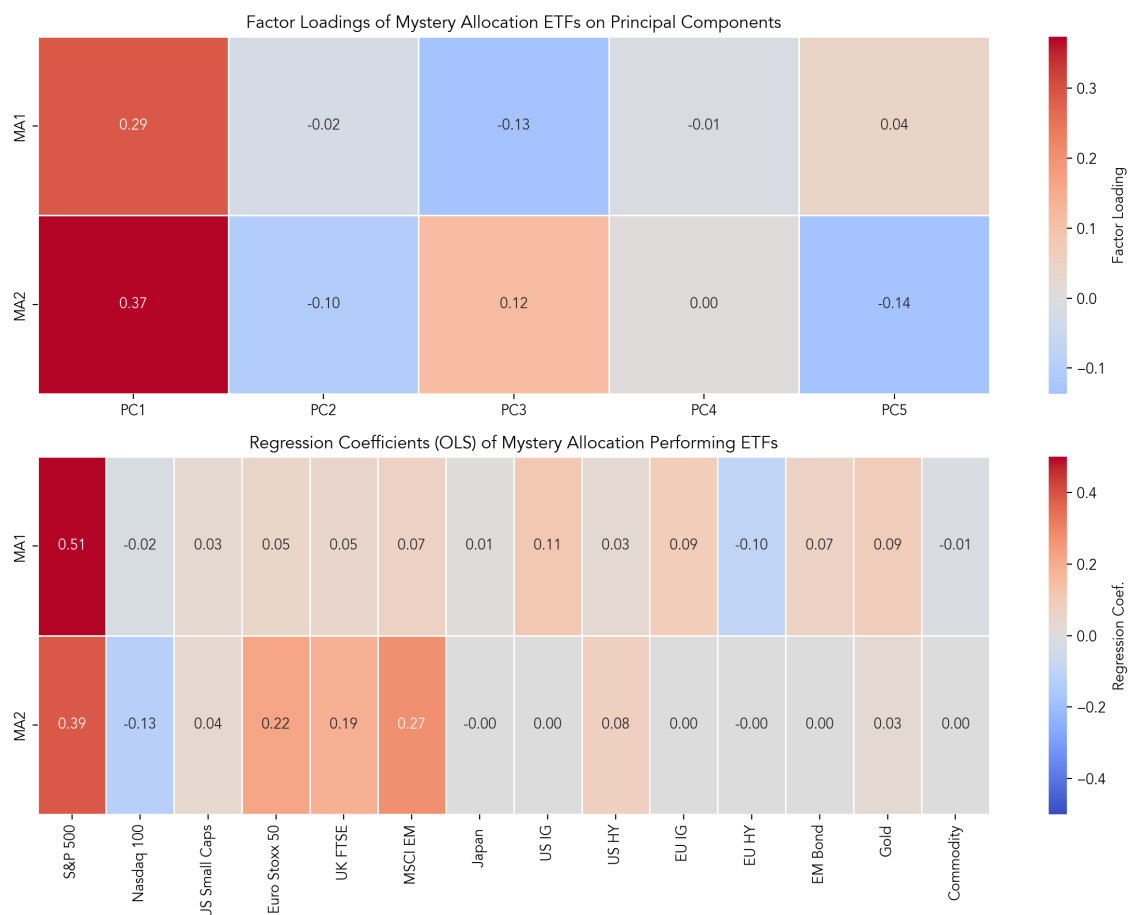


Figure 17. Asset classification for MAs.

Source: Own elaboration

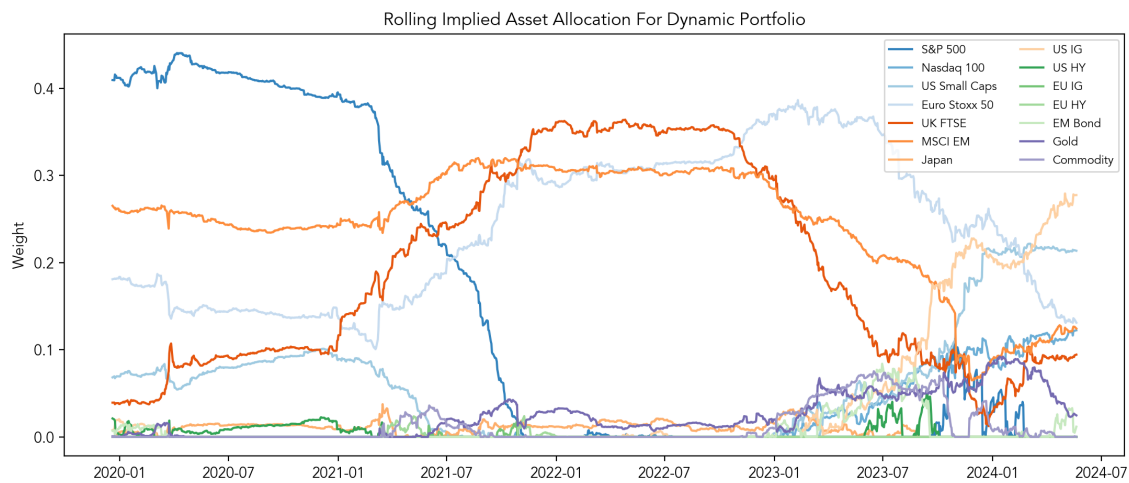


Figure 18. 15. Dynamic weights for MA2 (asset class based).

Source: Own elaboration

CONCLUSION

Our study integrated descriptive statistics, OLS regression, Principal Component Analysis and constrained optimisation to deconstruct the performance drivers of 101 anonymous ETFs and reverse-engineer two mystery portfolios over a volatile five-year period (2020–2024). Initial analysis of core performance metrics confirmed a classical risk-reward tradeoff, revealing a sharp division of the universe into two distinct clusters: low-volatility/low-reward defensive funds and high-volatility/high-return growth strategies.

This performance divergence was dictated primarily by structural asset allocation rather than idiosyncratic selection. The Top Performers delivered exceptional returns by aggressively isolating US Growth and Technology exposure, effectively riding the post-pandemic tech rally. In contrast, the Bottom Performers suffered a "safety penalty," heavily burdened by exposure to safe-haven assets like bonds and commodities, which acted as a drag during the inflationary regime.

The analytical framework culminated in the successful identification of the two mystery strategies. Mystery Allocation 1 (MA1) was revealed as a disciplined "Core-Satellite" approach that balanced permanent growth kickers (US, often tech equities) with investment-grade US/EU bonds. Mystery Allocation 2 (MA2) was identified as a geographically rotating, equity-focused portfolio, which pivoted from a defensive, value-oriented stance during the 2020–2022 volatility to a concentrated "Risk-On" bet on US Tech in late 2023.